with Mobile Backend 2
1. Setup SDK 3
2. Initialization 9
3. Register 12
4. Check POS Service 14
5. Start Transaction 15
6.Void, Refund, Report, Transaction History and Custom Request 24
7. Receipt Data 36
8. SoftPos Error 43
9. Pin Integration 46
10. SDK Flow Diagram 49
11. SoftPOS App Dependency List 50
12. TestUI Integration 53
13. Knox 55
14. Security Countermeasure 56
15. SoftPOS Runtime Error List 58
16.MF Qr Transaction Flow Implementation 69
17. QR Flow 76
18. MPoC Only API's 83
19. Device Custom Parameter (DCP) 84

with Mobile Backend

1. Setup SDK

1.1 Minimum System Requirements

SDK with PIN is compatible with Android OS 9 and later versions.

SoftPOS SDK is supported for devices that have NFC compatibility

1.2 Importing SoftPOS SDK

There are two options for importing SDK;

- Copying SoftPos SDK AAR lib into the libs folder and using it with the gradle below

Import Aar

```
allprojects {
   repositories {
      flatDir {
        dirs 'libs'
      }
   }
}

dependencies {
       implementation(name:'SoftPOS-v1.0.0', ext:'aar')
}
```

- Pulling SoftPos SDK over public Maven repository by using the gradle below

→ **username** and **password** informations which will be provided by Yazara and should be replaced in the gradle.

→ **implementation** gradle will also be provided by Yazara. Once the information is shared, only version part will be changed for future releases. You can add "-Release" or "-Debug" at the end of the version to get desired type(release,debug) of AAR.

Import Maven Aar

```
allprojects {
    repositories {
        maven {
            url 'https://softposrepo.paycore.com/repository/maven-releases'
            credentials {
                username = "yourUsername"
                password = "yourPassword"
            }
        }
        flatDir {
            dirs 'libs'
        }
    }
}

dependencies {
       implementation 'com.yazara:artifactId:1.0.0-Release@aar'
}
```

1.3 SoftPOS SDK Dependencies

SoftPOS SDK has some dependencies that need to be used in lifecycle. You can find the dependencies list below that should be added as libraries in the L3 UI/UX project.

Dependencies

```
implementation 'androidx.appcompat:appcompat:1.2.0'
implementation 'com.squareup.retrofit2:converter-moshi:2.6.4'
implementation 'com.squareup.retrofit2:converter-scalars:2.6.4'
implementation 'androidx.room:room-runtime:2.2.5'
annotationProcessor 'androidx.room:room-compiler:2.2.5'
implementation "com.google.android.gms:play-services-safetynet:17.0.0"
```

```
implementation 'com.flurry.android:analytics:14.0.0'
implementation 'com.google.android.gms:play-services-ads-identifier:18.0.1'
implementation 'androidx.appcompat:appcompat:1.3.0-alpha01'
```

Also, SoftPOS Debug SDK needs logging dependency which is specified below to show more logs in development phase.

Dependencies

```
implementation 'com.squareup.okhttp3:logging-interceptor:3.9.1'
```

1.4 Permissions

SoftPOS SDK needs some permissions using in lifecycle. Permissions are listed below. Permissions are not needed to be added in L3 UI/UX project, because SDK manifest file already has these permissions.

Permissions

```
<uses-permission android:name="android.permission.NFC" />

<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

<uses-permission android:name="android.permission.INTERNET" />

<uses-feature
    android:name="android.hardware.nfc"
    android:required="true" />
```

1.5 SoftPosService Class

SoftPosService class includes all API methods. SoftPosService class must be initialized to reach the API methods.

Initialize SoftPosService

```
SoftPosService softposService = new SoftPosService(Context context);
```

Table 1.5.1 SoftPos Service Parameters

Parameter Name	Description	Sample Value
context	Application context	

In case of any security problem detected by APK, SoftPosService has an API called "alertAppTamper" which can be used to wipe all the data inside SDK and crash the application after sending attestation to A&M Component.

Sample call is like below;

Sample Alert Call

Sample Alert Call

```
public void alertSDK() {
    this.softPosService.alertAppTamper();
}
```

1.6 SoftPosInfo Class

To use SoftPOS SDK below parameters needs to be set by SoftPosInfo class. You must set SoftPosInfto parameters before creating SoftPosService instance.

Table 1.6.1 SoftPos Info Parameter

Parameter Name	Description	Default Value
Url	Connection address for SoftPOS backend	
SafetNetApiKey	SoftPOS SDK uses Google Safetynet Api to check device integrity. Please visit SafetyNet Attestation API page and check "Obtain an API key"	
AcquirerId	Acquirer Identification for SoftPOS Backend internal usage	
ConnectionTimeoutSec	Connection timeout value	30 sec

Parameter Name	Description	Default Value
L2HostResponseTimeoutMs	The timeout value is to wait online response in L2Kernel. It means that whole transaction must be finished in a given timeout value.	180000 ms
HostCertificate	SoftPOS SDK uses Certificate Pinning during SSL connection. Therefore, it needs to your host certificate public key as a .cer format. This certificate must have valid CA sign, self signed certificate is not acceptable. Trusted cerfiticates could be root, intermediate or leaf certificate. If you call this method multiple times with different certificates, they will be added to valid host certificate list and will be used according to the validity. **This is mandatory**, you'll get IllegalArgumentException if it is not provided and application will crash.	
HostCertificateList	Description is the same as above. This method is provided to set all valid certificates at once.	
HostName	SoftPOS SDK uses Host Name Verification during SSL connection. Therefore, it needs to your host name value in the certificate. **This is mandatory**, you'll get ArgumentException if it is not provided and application will crash.	"*.paycore.com"
EnableForegroundDispatch	SoftPOS SDK can block all NFC triggers when application in foreground. This feature is disabled in default.	
IsoDepTimeoutMs	NFC connection timeout value. In some VpTT test cases of L3 certification, you need to set it 15000 for tool compatibility.	2000 ms
EnableFileLogging	It is used to enable file logging. You can also pass a custom path parameter to store the log file in this path. File logging is disabled in default. Ps. Do not forget if you set a path to need "android.permission.WRITE_EXTERNAL_STORAGE" permission, you must grant it from user before creating SoftPosService instance. And you must add this line in your AndroidManifest.xml `android:requestLegacyExternalStorage="true"` In addition, please make sure that the storage permission in settings→your App→permissions is allowed.	Default Path : /data/data/packageName/files/SoftPosLog/yyyyMMdd_SoftPosSDK.log
DisableFileLogging	It is used to disable file logging.	
EnableGeoCoordinates	It is used to enable the location requirement of Softpos SDK. Geo coordinates is disabled in default.	
DisableGeoCoordinates	It is used to disable geo coordinates.	
DisableCookieManager	It is used to disable the cookies used by an application	
EnableCookieManager	It is used to enable the cookies used by an application.	
DisablePinAppAutoKill	Its is used to disable when the pin entry process is finished, the automatically close pin app	
EnablePinAppAutoKill	Its is used to enable when the pin entry process is finished, the automatically close pin app	
DisablePinOnRefund	Its is used to disable pin on refund.	
ForceAmountDecimalSeparator	A decimal separator is a symbol used to separate the integer part from the fractional part of a number written in decimal form. It is used set the decimal separator symbol.	
ForceAmountThousandSeparator	ThousandSeparator is used to separate groups of thousands in floating point numbers or currencies. It is used set the thousand separator symbol.	
PayloadStoreTimeoutMs		
FileNameId		
PinAppLanguage	It is used for setting pin application language	
PinAppPackageName	Application Package Name it should be unique. its used for triggering the Pin Application	
PinAppOrientation	PinApp screen orientation set parameter. Default is portrait.	
UiName		

Parameter Name	Description	Default Value
UiVersion		
TestUICert		
TestUIIP		
TestUIPort		
SetLogApiDisable	Send logs flurry to enable or disable	
DetectionListener	When the SDK detects a security vulnerability, this callback is called and the application closes in a random time between 2 and 4 seconds.	

An exemplary usage is below:

Setting Program Parameters

```
SoftPosInfo.setUrl("https://softpostest.paycore.com/coreV2");

SoftPosInfo.setSafetyNetApiKey("AIzaSyC0Zt-_YOsnlc4vGG4ZGRpT53D36Zl3_FY");

SoftPosInfo.setAcquirerId(766573);

SoftPosInfo.setConnectionTimeoutSec(60);

SoftPosInfo.setHostCertificate(hostCertificate);

SoftPosInfo.setHostName("*.paycore.com");

SoftPosInfo.enableForegroundDispatch();

SoftPosInfo.setL2HostResponseTimeoutMs(180000); //3 min is the max value. I'll not get any higher if greater value i

SoftPosInfo.setIsoDepTimeoutMs(15000);


SoftPosInfo.enableFileLogging(); // use default log path

SoftPosInfo.enableFileLogging("/sdcard/download"); // set custom log path

SoftPosInfo.disableFileLogging();

SoftPosInfo.enableGeoCoordinates();




SoftPosInfo.setDetectionListener(new DetectionListener() {
    @Override
    public void onRiskDetected() {
        new Handler(Looper.getMainLooper()).post(() -> {
            Log.e("onRiskdetected", "An Exception Occurred");
        });
   }
});
```

1.7 Enable Foreground Dispatch

SoftPosInfo.enableForegroundDispatch method is only used to enable foreground dispatch of NFC for given activity on startTransaction method. It is activated after calling startTransaction and it is enabled during your activity lifetime. If you want to enable it when you activity created, please use enableForegroundDispatch method in SoftPosService instead of SoftPosInfo class.

This method must be called from the main thread, and only when the activity is in the foreground (resumed). Also, activities must call disableForegroundDispatch(Activity) before the completion of their Activity#onPause callback to disable foreground dispatch after it has been enabled.

Setting Program Parameters

```
public boolean enableForegroundDispatch(Activity activity)
```

Setting Program Parameters

```
public boolean disableForegroundDispatch(Activity activity)
```

Table 1.6.1 Enable and Disable Foreground Dispatch Parameters

Parameter Name	Description
activity	The activity of the application. It is used to dispatch NFC when your activity is foreground.

1.8 Attestation Service

SoftPos SDK ensures integrity of application and device health check with SoftPOS Attestation Service in addition to Google SafetyNet. You must provide required identifications for SoftPOS Attestation Service which are specified below to use SoftPOS SDK successfully.

Table 1.8.1 Required Parameters

Field Name	Descriptions	Sample Value
Model of the device	List of allowed device models	SM-N950F
Manufacturer of device	List of allowed device manufacturers	Samsung
OS Versions	List of allowed OS versions	9
Package name of App	List of allowed applications' package names	com.softpos.app
Version of App	List of allowed applications' versions	1.0.0
Permission List of App	List of allowed applications' permissions	"android.permission.INTERNET", "android.permission.ACCESS_WIFI_STATE", "android.permission.ACCESS_NETWORK_STATE", "android.permission.NFC", "android.permission.ACCESS_FINE_LOCATION", "android.permission.ACCESS_COARSE_LOCATION" "android.permission.WRITE_EXTERNAL_STORAGE", "android.permission.READ_EXTERNAL_STORAGE"
Digest(SHA256) of Signing Certificate	List of allowed applications' SHA256 value of signing certificate. It should be hex value.	783DD524F0FD284FDD93F616D380E4BEC79910E86CAA5AC1144ACA5BAE7AE25D

1.9 Embedded Pin Requirements

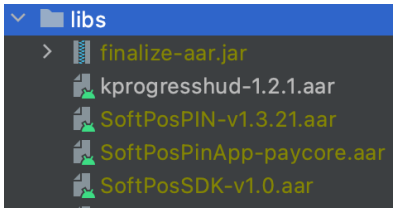
SoftPos SDK supports embedded pin solution. Therefore, below changes should be applied to use SoftPos SDK with embedded pin.

Embedded PIN aar should be added to dependency list of SoftPos Application.

Please notice that minimum android SDK version is bumped up from 19 to 25 for SDK to be work as embedded in application.

```
/** SoftPos SDK */
implementation (name:'SoftPosSDK-v1.0', ext:'aar')
/** SoftPos PIN SDK */
implementation (name:'SoftPosPIN-v1.3.21', ext:'aar')
/** SoftPos Embed PIN */
implementation (name:'SoftPosPinApp-paycore', ext:'aar')
```

PinApp aar lib should be copied under libs folder (where SoftPosSDK and PinSDK is also located)



Gson dependency should be added to dependency list of SoftPos Application

```
implementation 'com.google.code.gson:gson:2.8.6'
```

PinApplicationPackageName property in SoftPosInfo should be set as *"context.aar"* and pinAppVersion should be filled like below.

```
SoftPosInfo.setPinAppPackageName("context.aar");
SoftPosInfo.setPinAppVersion(com.softpos.pin.BuildConfig.VERSION_NAME);
```

After all the changes above are applied, embedded pin app solution is ready to work.

1.10 Applying finalizer

Finalization is a new required process for SDK to have higher security standards.

If you don't run below command by using the 'finalizer executable' for your apk/aab output, your application will crash due to the usage of unfinalized AAR's.

./finalizer -i unfinalized.apk -f finFile1.fin,finFile2.fin -o finalizedApkFolder

./zipalign -p -v 4 finalizedApkFolder/unaligned_unsigned.apk finalizedApkFolder/aligned_unsigned.apk

./apksigner sign --ks ~/example.jks --ks-pass 'pass:yourPassword' --out finalizedApkFolder/aligned_signed.apk finalizedApkFolder/aligned_unsigned.apk

After all the changes above are applied, finalization will be completed. Please be patient because will take some time to complete.

Finalize executable will externally be shared by Yazara.

IMPORTANT : Below values should be defined in global environment variables. Otherwise builds will fail.

JAVA_HOME="JAVA PATH IN YOUR MACHINE"
ANDROID_HOME="ANDROID SDK PATH IN YOUR MACHINE"

1.11 Excluding Special Processes

Special processes which are created by SDK can lead to a problem according to the implementation of the android application class.

It is recommended to in application onCreate method and DO NOT take any action if the process is defined as SDK specific process.

An example call in Application class should look like below;

Application

```
@Override
public void onCreate() {
    super.onCreate();

    //if the process is not application default one, then do not take any specific action.
    if(ProcessCheckUtil.isSpecialProcess(this)){ return; }

    //your application code
    //..
    //..
}
```

2. Initialization

2.1 API Description

This method is the initialization API of the SDK. This is a prerequisite step for most of other SDK methods / API calls, which means until getting onPOSReady response of this function, initialization method should be called after register and getting permission step.

Initialize

```
public void initialize(final InitializeListener initializeListener)
```

2.2 InitializeListener

The response of the initialization method can be reached using InitializeListener methods. An exemplary usage is below:

InitializeListener

```
public interface InitializeListener {

    void onPOSReady();

    void onRegisterNeed();

    void onPermissionNeed(ArrayList<String> missingPermission);

    void onInitializeError(SoftPosError error);

}
```

Table 2.2.1 Initialize Listener Methods

Method Name	Description	Action
onPOSReady	SDK is ready to make transaction	Call CheckPOSService method before starting transaction. Please check 4. Check POS Service section.
onRegisterNeed	SDK needs to register device on backend server to create device unique credential.	Call Register method. Please check 3. Register section. After successful registration call once again initialize method to get onPOSReady response.
onPermissionNeed	Missing permission list of SDK is returned as String parameters as "android.permission.ACCESS_FINE_LOCATION".	Ask permission request to user. After getting permission confirmation use initialize method to get onPOSReady response.
onInitializeError	When SDK encounters unhandled error case during initializing library, this method will be triggered with the error message.	Get error detail using SoftPosError class. Please check 6. SoftPos Error section.

2.3 Deinitialization

This method is the deinitialization API of the Kernel. If you want to start transaction after calling this method, you have to call initialize method before calling startTransaction method.

Initialize

```
public void boolean deInitialize()
```

2.4 Get Version

This method is used to get SDK version.

Versioning methodology is like;

Parts 1-2-3 are for source code versioning. 4th part represents customer and the last(5th) part represents public key version

Example : 1.2.229.3.10

1.2.229 → Source Code version

x.x.x.3.x → Customer identification number

x.x.x.x.10 → 10th version of public keys. If renewal period is monthly then 10 means October keys.

getVersion

```
public void String getVersion()
```

2.5 Get Kernel Version & Commit ID

This method is used to get L2Kernel version and commit id values for requested L2 Kernel.

getKernelVersion

```
public KernelVersion getKernelVersion(CardType cardType)
```

2.5.1 Card Type

The getKernelVersion needs Card Type identifier to get specific version and commit id values for requested L2 Kernel

CardType

```
public enum CardType {
    None(0),
    JCBAndVisa(1),
    MasterCard(2),
    Visa(3),
    Amex(4),
    JCB(5),
    Discover(6),
    UnionPay(7),
    Rupay(13);
}
```

2.5.2 Kernel Version

The response of the getKernelVersion method can be reached using KernelVersion methods. An exemplary usage is below:

KernelVersion

```
public class KernelVersion {

    public String getVersion();

    public String getCommitID();
}
```

Table 2.5.1 Kernel Version Methods

Parameter Name	Description	Sample Value
getVersion	Version of given L2 Kernel identifier	3.1.2
getCommitID	Commit ID value of given Card Type identifier	`da1e341e451a93593167f8476a1f656844379c96`

2.6 Get Currency

This method is used to get currency code from POS parameters. You must call this method after registration. The return value of the method is prepared according to **ISO 4217** standard

getCurrencyCode

```
public void CurrencyCode getCurrencyCode()
```

CurrencyCode

```
public enum CurrencyCode {

    public String name();
```

```
    public String getCode();

    public String getDesc();
}
```

Table 2.5.1 Currency Code Methods

Method Name	Description	Sample Value
name	It is used to get code value of currency according to ISO 4217	EUR
getCode	It is used to get number value of currency according to ISO 4217	978
getDesc	It is used to get description of the currency	Euro

2.7 Get NFC Location Gif URL

This method is used to get Web URL for a GIF which shows the NFC antenna location on the current device. You can just redirect the user to the link provided by browser or web view in the application. It can be used via SoftPosService

CurrencyCode

```
public String getNFCLocationGifUrl()
```

3. Register

3.1 API Description

Upon downloading the mobile app, one of the initial steps to be completed is to register. Registering the SDK requires some merchant specific parameters to be passed by the mobile app, which will then be associated with this instance of the SDK by the backend.

Register

```
public void register(String merchantId, String terminalId, String activationCode, RegisterListener registerListener
```

Table 3.1.1 Register Parameters

Parameter Name	Description
merchantId	Merchant identifier is used to validate in IDNV service
terminalId	Terminal identifier is used to validate in IDNV service
activationCode	Activation code is used to validate in IDNV service
registerListener	Asynchronous caller function.

3.2 Register Listener

The response of the register method can be reached using RegisterListener methods. An exemplary usage is below:

RegisterListener

```
public interface RegisterListener {

    void onRegisterSuccess();

    void onRegisterError(SoftPosError error);
}
```

Table 3.2.1 Register Listener Methods

Method Name	Description	Action
onRegisterSuccess	Registration is finished successfully.	Call initialize method to get onPOSReady response. Please check 2. Initialization section
onRegisterError	When SDK encounters unhandled error during registration, this method will be triggered with the error message.	Get error detail using SoftPosError class. Please check 8. SoftPos Error section.

3.3 Unregister

In order to make new registration, unregister method should be called to remove old user info in device.

Register

```
public void unregister(UnregisterListener unregisterListener)
```

Table 3.3.1 Unregister Parameters

Parameter Name	Description
unregisterListener	Asynchronous caller function.

3.4 Unregister Listener

The response of the unregister method can be reached using UnregisterListener methods. An exemplary usage is below:

UnregisterListener

```
public interface UnregisterListener {

    void onUnregisterSuccess();

    void onUnregisterError(SoftPosError error);
}
```

Table 3.4.1 Register Listener Methods

Method Name	Description	Action
onUnregisterSuccess	Unregistration is finished successfully.	Call register method for a new user.
onUnregisterError	When SDK encounters unhandled error during unregistration, this method will be triggered with the error message.	Get error detail using SoftPosError class. Please check 6. SoftPos Error section.

3.5 Register Error

If you are getting the error ClassNotFoundException: Didn't found class "com.softpos.pin.sdk" in register, please implement the pin sdk and install the pin application on your device.

4. Check POS Service

4.1 API Description

This is a general status check API for SDK, which can be called before every transaction on the mobile app level. Upon calling this API, the SDK checks various conditions to get ready for transaction.

CheckPOSService

```
public void checkPOSService(final CheckPOSServiceListener checkPOSServiceListener)
```

4.2 Check POS Service Listener

The response of the checkPOSService method can be reached using CheckPOSServiceListener methods. An exemplary usage is below:

CheckPOSServiceListener

```
public interface CheckPOSServiceListener {

    void onCheckPOSSuccess();

    void onCheckPOSError(SoftPosError error);
}
```

Table 4.2.1 Check POS Service Methods

Method Name	Description	Action
onCheckPOSSuccess	Status check is finished successfully. SDK is ready to start a transaction.	Call start transaction method to make transaction. Please check 5. Start Transaction section
onCheckPOSError	When SDK encounters unhandled error during checking SDK status, this method will be triggered with the error message.	Get error detail using SoftPosError class. Please check 8. SoftPos Error section.

5. Start Transaction

5.1 API Description

This method is used to start a contactless EMV transaction. At this point of time, it is assumed that the app already has the sales amount fixed, and the contactless logo is displayed on the screen so that the consumer understands that he or she needs to tap the contactless payment card to the SoftPOS device.

Calling this API will trigger the SDK to enable NFC for a contactless communication with the payment card. If a successful transaction occurs between the two, then SDK submits the transaction to acquiring backend for online authorization.

StartTransaction

```
public void startTransaction(final long amount, final int timeoutMs, final Activity activity, final TransactionList
```

StartTransaction

```
public void startTransaction(final long amount,  final TransactionType transactionType, final int timeoutMs, final
```

StartTransaction

```
public void startTransaction(final long amount, final TransactionType transactionType, final int timeoutMs, final S
```

StartTransaction

```
public void startTransaction(final long amount, final TransactionType transactionType, final int timeoutMs, final S
```

Table 5.1.1 Start Transaction Parameters

Parameter Name	Description	Sample Value
amount	Amount of transaction	Set 1300 for 13 € (or other currencies)
transactionType	Emv parameter of the transaction	• SALE • REFUND • VOID
timeoutMs	Timeout value to tap the card to the mobile device.	10000 ms
mti	Type of Transaction (Sale, Void vs)	**Type** / **Value** Sale: 0100 Void: 0200 Refund: 0300 Void Cancellation: 0400 Pre Authorization: 0500

customMessage	Custom message field in transaction request	In void and refund transaction please send transactionId field in customMessage parameter.
		TransactionID
		`Map<String, String> customMessage = new HashMap<>();`
		`message.put("transactionId", "17823678");`

activity	Activity of the transaction fragment. It is used to enable NFC for this activity.	
transactionListener	Asynchronous caller	

Parameter Name	Description	Sample Value
	function.	

cardAppInfo	Object to filter the transaction card applications. This parameters is optional. For class details please see section 5.10	**TransactionID**
		`CardAppInfo cardAppInfo = new CardAppInfo();`
		`cardAppInfo.setCardType("C");`
		`cardAppInfo.setAID("A0000000031010");`

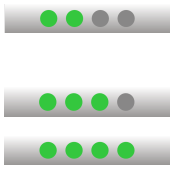
5.2 Transaction Listener

The response of the startTransaction method can be reached using TransactionListener methods. An exemplary usage is below:

TransactionListener

```
public interface TransactionListener {

    void onCardDetected();

    void onCardReadSuccess();

    void onCardReadFail();

    void onGoOnline(CardType cardType);

    void onPlaySound(BeepType type);

    void onCompleted(TransactionResult result);

    void onTimeout();

    void onStartTransactionError(SoftPosError error);

    void onCardStillExists();

    void onCardRemoved();
}
```

Table 5.2.1 Transaction Listener Methods

Method Name	Description	Action	Sample POS Led Design
onCardDetected	It is triggered when SDK can start reading the card	Inform user that mobile app can detect the card and reading process continues. First led should be turned on.	
onCardReadSuccess	It is triggered when SDK has finished reading the card	Inform user that mobile app has finished reading the card. User can remove the card from mobile phone. Three other leds should be turned on in 250 ms and stay 750 ms, after that all leds should be turned of.	
onCardReadFail	It is triggered when SDK gets any error while reading the card	Inform user that mobile app could not read the card. User can tap the card again to finish transaction.	
onGoOnline	It is triggered when SDK has prepared transaction data for online authorization	Inform user that mobile app is sending transaction data to get online authorization.	
onPlaySound	It is triggered when L2Kernel wants to play beep sound.	Check BeepType value and play success of fail type sound	
onCompleted	It is triggered when SDK has finished transaction.	Check TransactionResult and inform user that transaction has been accepted or failed.	

Method Name	Description	Action	Sample POS Led Design
onTimeout	It is triggered when SDK could not detect any cards in a given timeout value	Inform user that mobile app could not detect any cards. Transaction is cancelled.	
onStartTransactionError	It is triggered when SDK gets an error during starting transaction	Check the SoftPosError object to understand the error.	
onCardStillExists	It is triggered when user did not remove card after tag detected	Inform user that card should be removed to continue.	
onCardRemoved	It is triggered when user removed the card	Inform user that transaction continues e.g Please wait or Authorizing	

5.3 Beep Type

L2 Kernel has to inform that transaction is completed by playing a sound. It is mandatory for L2 Kernel certification. There are two sound types to inform the consumer. Check BeepType value and play sound when onPlaySound method is triggered.

BeepType

```
public enum BeepType {
    SUCCESS, FAIL, WARN
}
```

5.4 Card Type

L2 Kernel returns Kernel ID value which is identifier for Card Type.

BeepType

```
  public enum CardType {
    None(0),

    JCBAndVisa(1),

    MasterCard(2),

    Visa(3),

    Amex(4),

    JCB(5),

    Discover(6),

    UnionPay(7),

    Rupay(13),

    Pure(45);
}
```

5.5 Transaction Result

When transaction is completed, onCompleted method will be triggered. Result of the transaction can be received by using TransactionResult methods.

TransactionResult

```
public class TransactionResult {

    public boolean isEMVAccepted();

    public String getRefusalCode();
```

```
    public String getMaskedPan();

    public CardType getCardType();

    public String getIssuerIdentificationNumber();

    public Map<String, String> getCustomMessage();

    public String getOutcomeMsgId();

    public String getL2ErrorIndication();

    public String getTransactionId();

    public String getApplicationLabel();

    public String getApplicationPreferredName();

    public String isSCANeeded();

    public SoftPosError getTransactionError();
}
```

Table 5.5.1 Transaction Result Methods

Parameter Name	Description	Sample Value
isEmvAccepted	L2 Kernel has completed the transaction successfully.	true or false
getRefusalCode	The authorization result of the transaction	51 - (insufficient fund)
getMaskedPan	Last 4 digit of the card	7204
getCardType	Kernel Id value of the transaction	CardType.MasterCard
getIssuerIdentificationNumber	Issuer Identification Number of card	534567
getCustomMessage	Response message of your custom message in transaction request	
getOutcomeMsgId	Kernel message index. It is mapped on "Table 9-5: User Interface Standard Messages" section at EMV Book A	1B
getL2ErrorIndication	Kernel error indication	00
getTransactionId	The unique identifier of the transaction.	929616000304
getApplicationLabel	The application label field of the card. It can be empty depending on card.	Debit MasterCard
getApplicationPreferredName	The application preferred name of the card. It can be empty depending on card.	NEO
isSCANeeded	Start new transaction for SCA re-tap workflow. Please check section 5.9 Strong Customer Authentication	true or false
getTransactionError	If you get a system error which refusal code is 96, you can get the error details with using this method. Please check 8. SoftPos Error section for the details. It will be null if the authorization host return 96 value.	

Table 5.5.2 Sample Transaction Result Values

Sample Value	Meaning
isEmvAccepted = false, refusalCode = "", maskedPan = "", cardType = CardType.None, issuerIdentificationNumber = "", outcomeMsgId = "18", l2ErrorIndication = "00", transactionId = null, applicationLabel = "", applicationPreferredName = "", isSCANeeded = false	Kernel Decline Transaction
isEmvAccepted = true, refusalCode = "51", maskedPan = "7204", cardType = CardType.MasterCard, issuerIdentificationNumber = "534567", outcomeMsgId= "1B", l2ErrorIndication = "00", transactionId = "929616000304", applicationLabel = "Debit MasterCard", applicationPreferredName = "NEO", isSCANeeded = false	Kernel Accept, but Online Host Decline Transaction
isEmvAccepted = true, refusalCode = "00", maskedPan = "7204", cardType = CardType.MasterCard, issuerIdentificationNumber = "534567", outcomeMsgId = "1B", l2ErrorIndication = "00", transactionId = "929616000304", applicationLabel = "Debit MasterCard", applicationPreferredName = "NEO", isSCANeeded = false	Kernel and Online Host Accept Transaction

5.5.3 Transaction Result Flow

5.6 Cancelling a Transaction

cancelTransaction method is used to cancel listening the card and disable NFC reader. It is recommended calling this function with onStop and onDestroy methods of fragment or activity which startTransaction is called. If cancelTransaction is called during sending online transaction request, it will return false because Kernel has finished transaction and SDK is getting online verification. The transaction cannot be cancelled in this process.

StartTransaction

```
public boolean cancelTransaction()
```

5.7 Void Transaction Without Tapping Card

This method is used for void operation given transaction id without tapping card.

5.7.1 Start Void Transaction

VoidTransaction

```
public void startVoidTransaction(final String transactionId, final CardlessTransactionListener cardlessTransactionL
```

Table 5.7.1 Start Void Transaction Parameters

Parameter Name	Description	Sample Value
transactionId	Transaction unique id	Transaction unique id
cardlessTransactionListener	Asynchronous caller function.	

5.7.2 Cardless Transaction Listener

The response of the startVoidTransaction method can be reached using CardlessTransactionListener methods. An exemplary usage is below:

CardlessTransactionListener

```
public interface CardlessTransactionListener {

    void onCardlessTransactionSuccess(CardlessTransactionResult cardlessTransactionResult);

    void onCardlessTransactionError(SoftPosError softPosError);
}
```

Table 5.7.2 CardlessTransactionListener Methods

Parameter Name	Description	Action
onCardlessTransactionSuccess	Void operation is finished.	Check CardlessTransactionResult and inform user that transaction has been accepted or failed.
onCardlessTransactionError	When SDK encounters unhandled error during void operation, this method will be triggered with the error message.	Get error detail using SoftPosError class. Please check 8. SoftPos Error section.

5.7.3 Cardless Transaction Listener

When void transaction is completed, onCardlessTransactionSuccess method will be triggered. Result of the void transaction can be received by using CardlessTransactionResult methods.

CardlessTransactionResult

```
public class CardlessTransactionResult {

    public String getRefusalCode();

    public Map<String, String> getCustomMessage();

    public String getTransactionId();
}
```

Table 5.7.3 Transaction Result Methods

Parameter Name	Description	Sample Value
getRefusalCode	The authorization result of the transaction	00 - Success
getCustomMessage	Response message of your custom message in transaction request	
getTransactionId	The unique identifier of the transaction.	929616000304

5.7.4 Void Transaction Without Tapping Card Result Flow

5.8 Get Tag Value

This method is used to get a tag value from L2 Kernel after transaction. It returns data on TLV format. You must pad your parameter tag value with FF. An exemplary usage is below.

getTagValue

```
public byte[] getTagValue(byte[] bytes)
```

5.8.1 Sample Usage

getTagValue

```
byte[] result = softPosService.getTagValue(new byte[]{ (byte)0x9F, (byte)0x12, (byte)0xFF });
```

Sample Parameter	Sample Result
9F12FF	9F12034E454F
50FF	50104465626974204D617374657243617264
9F1AFF	9F1A020792

5.9 Strong Customer Authentication (SCA)

Strong customer authentication (SCA) is a requirement on the European Union countries. If you do not support this feature please skip this part.

SoftPos SDK can handle SCA one tap work flow internally. For the re-tap workflow, you need to check isSCANeeded flag on TransactionResult object and start new transaction with setting isSCATransaction parameter as true on startTransaction method if isSCANeeded is true.

5.9.1 Transaction Result

When transaction is completed, onCompleted method will be triggered. Result of the transaction can be received by using TransactionResult methods.

TransactionResult

```
public class TransactionResult {

    public String isSCANeeded();
}
```

Table 5.9.1 Transaction Result Methods For SCA

Parameter Name	Description	Sample Value
isSCANeeded	Start new transaction for SCA re-tap workflow.	true or false

5.9.2 Transaction Result Flow With SCA

5.9.3 Start Transaction API for SCA

For the SCA re-tap workflow, you need to start new transaction with the same parameters and passing isSCATransaction parameters as True. The following steps are the same with the original startTransaction flows.

StartTransaction

```
public void startTransaction(final long amount, final int timeoutMs, final boolean isSCATransaction, final Activity
```

StartTransaction

```
public void startTransaction(final long amount,  final TransactionType transactionType, final int timeoutMs, final
```

StartTransaction

```
public void startTransaction(final long amount, final TransactionType transactionType, final int timeoutMs, final S
```

Table 5.9.2 Start Transaction Parameters For SCA

Parameter Name	Description	Sample Value
isSCATransaction	It defines that you start a transaction for SCA re-tap workflow.	true or false.

5.10. Single Tap / Double Tap

SingleTap and DoubleTap features are included in SoftPosSDK and this section is to provide integration details for the implementation.

5.10.1 CardAppInfo object

First of all, CardAppInfo object is needed to be mentioned to explain the implementation details.

CardAppInfo

```
public class CardAppInfo {

    private int kernelId;

    private String AID;

    private String cardType;
```

```
    private String appLabel;

    private String trnxType;
}
```

Parameter	Description	Sample Value
kernelId	kernel identification parameter to be used for filtering applications according to the kernel value.	Integer value of CardType enum in section 5.4
AID	Application ID parameter to be used for filtering applications according to AID value	"A0000000003010"
cardType	Card type parameter to be used for filtering applications according to the card type like Debit/Credit	"C" : Credit "D" : Debit
appLabel	Application Label parameter to be used for showing defined appLabel to the user as text. This parameter is not used for any filtering.	"Mastercard Debit"
trnxType	Transaction type parameter to be used for filtering applications according to type of transaction	TransactionType.SALE.toValueStr();

5.10.2 Single Tap

SDK supports transactions to start with specific information or filter. This mechanism is called "Single Tap" which allows transactions to start with specific card information and it is mostly used by combo cards(multiple application cards).

SDK has an api called ***"prepareAppList"*** which returns list of ***"CardAppInfo"*** object for supported card applications in terminal. This API could be used to show supported applications to user. For instance, It can be used for listing supported AID's or Debit/Credit selection.

It is not required to call this method to make single tap transactions. CardAppInfo object can be created externally by setting the object fields like AID, CardType etc.. SDK will start a card app filtered transaction with respect to the cardAppInfo object properties.

Example usage is like below which filters only credit card applications;

StartTransaction

```
CardAppInfo cardAppInfo = new CardAppInfo();
cardAppInfo.setCardType("C");

SoftPosApp.getSoftPosService().startTransaction(1000L, TransactionType.SALE,20000, "0100", null, false, getActivity
```

Another example usage is getting CardAppInfo via SoftPosSDK.

StartTransaction

```
List<CardAppInfo> cardAppInfoList = prepareAppList(TransactionType.SALE); //Prepares cardAppInfo list according to
CardAppInfo cardAppInfo = cardAppInfoList.get(0);

SoftPosApp.getSoftPosService().startTransaction(1000L, TransactionType.SALE,20000, "0100", null, false, getActivity
```

5.10.3 Double Tap

SDK supports transactions with multiple AID cards. If terminal include the applications that combo card has then SDK will trigger the callback ***"onDoubleTapNeeded"*** by passing the matching application list as parameter. In this case, application should let the user select the desired application. After the user made his/her choice, selected cardAppInfo object should be passed to startTransaction api as parameter. From this point, transaction will be considered as the same as "single tap" approach.

Please notice that double tap permissions setting is needed to defined in merchant/terminal configuration as "force double tap" or "double tap" to be able to use this feature.

Setting doubleTapListener instance is required before starting the transaction to be able to receive onDoubleTapNeeded callback. If not below error will be returned from SDK.

SDK 132 Error

```
E132(132,"Required transaction listener is not provided"),
```

DoubleTapListener

```
public interface DoubleTapListener {

    void onDoubleTapNeeded(List<CardAppInfo> candidateList);
```

```
}
```

Parameter	Description
onDoubleTapNeeded	It is triggered when card application selection is needed if the card has multiple applications matches with the included ones in terminal. Candidate list is provided as parameter which includes selectable card applications.

Example usage is like below;

StartTransaction

```
CardAppInfo selectedAppInfo = new CardAppInfo();

SoftPosApplication.getSoftPosUtil().getSoftPosService().setDoubleTapListener(new DoubleTapListener() {
    @Override
    public void onDoubleTapNeeded(List<CardAppInfo> list) {

        //Open a pop-up or screen to let the user make the choice.
        selectedAppInfo = list.get(0); //let's consider first item is selected

        //checkPosService call is still required here as it is in normal transaction.
        SoftPosApplication.getSoftPosUtil().checkPOSService(this);
    }
});


@Override
public void onCheckPOSSuccess() {
    //start a new transaction by passing the selected application object as parameter
    SoftPosApp.getSoftPosService().startTransaction(1000L, TransactionType.SALE,20000, "0100", null, false, getActi
}
```

6.Void, Refund, Report, Transaction History and Custom Request

6.1 Send Custom Request

This method is used to send custom requests to SoftPos backend on secure channel. Also it is used to list void and refund transactions.

sendCustomRequest

```
public <E,T> void sendCustomRequest(final RequestType requestType, final E request, final Class responseClassType,
```

sendCustomRequest

```
public <E,T> void sendCustomRequest(final String path, final E request, final Class responseClassType, final Custom
```

Table 6.1.1 Custom Request Parameters

Parameter Name	Description	Sample Value

requestType	Type of request	`RequestType.GET_VOID_LIST,`
		`RequestType.GET_REFUND_LIST,`
		`RequestType.GET_REPORT,`
		`RequestType.SEND_TO_HOST`

path	In order to make request to custom sub domains, set path value. It is added to the main url which is set via SoftPosInfo.setUrl method.	"getVoid"
request	Instance of request class. The class should have @Json annotation for related field. Note: To send raw json, set json string.	Check Example 6.1.1 Request Class
responseClassType	Class type of response object. It is required for deserialize response to given class type. Note: To get response as a json string. Set String.Class	ListResponse[].class
customRequestListener<T>	Asynchronous caller function. Note: To get response as a json string. Set CustomRequestListener<String>	

Example 6.1.1 Request Class

ListRequest

```
public class ListRequest {

    @Json(name = "transactionId")
    public String transactionId;
```

```
    @Json(name = "page")
    public Integer page;

    @Json(name = "pageCount")
    public Integer pageCount;
}
```

Example 6.1.2 Response Class

ListResponse

```
public class ListResponse {

    @Json(name = "transactionId")
    public String transactionId;

    @Json(name = "stan")
    public String stan;

    @Json(name = "transmissionDate")
    public Integer transmissionDate;

    @Json(name = "transmissionTime")
    public Integer transmissionTime;

    @Json(name = "amount")
    public Integer amount;
}
```

6.1.1 Custom Request Listener

The response of the getVoidList method can be reached using VoidListListener methods. An exemplary usage is below:

CustomRequestListener

```
public interface CustomRequestListener<T> {

    void onGetCustomResponse(T response);

    void onCustomRequestError(SoftPosError error);
}
```

Table 6.1.2 Custom Request Listener Methods

Method Name	Description	Action
onGetCustomResponse	Response is successfully gotten.	
onCustomRequestError	When SDK encounters unhandled error during sendCustomRequest call, this method will be triggered with the error message.	Get error detail using SoftPosError class. Please check 8. SoftPos Error section.

Example 6.1.3 Method Usage

SendCustomRequest

```
ListRequest listRequest = new ListRequest();

listRequest.page = 1;

listRequest.pageCount = 5;

listRequest.transactionId = "123123";

SoftPosApp.getSoftPosService().sendCustomRequest(RequestType.GET_VOID_LIST, listRequest, ListResponse[].class, new
    @Override
```

```
    public void onGetCustomResponse(ListResponse[] result) {

    }

    @Override
    public void onCustomRequestError(SoftPosError error) {
    }
});
```

6.2 Void List

In order to list void transaction, sendCustomRequest method must be used. You must create your request and response classes. You can find the expected request and response for get void list request below.

6.2.1 Void List Request

VoidListRequest

```
{"transactionId":"123123","amount":1500,"secondA    mount":1500,"amountType":1,"authorizationId":"123456","batchNum
```

Table 6.2.1 Void List Request Parameters

Parameter Name	Parameter Type	Description	Sample Value
transactionId	Optional	Transaction unique id	100004892014
amount	Optional	Transaction amount value	100 for 1,00
secondAmount	Optional	Transaction amount value	100 for 1,00
amountType	Optional	Add amount query with below parameter. **Value / Type / Description** 0 / Empty / no filter is applied 1 / LessThan / less than given amount 2 / LessThanOrEqualTo / less than or equal togiven amount 3 / GreaterThan / greater than given amount 4 / GreaterThanOrEqualTo / greater than or equal to given amount 5 / GreaterThanOrEqualTo & LessThanOrEqualTo / greater or equal to amount and less or equal to secondAmount	
authorizationId	Optional	Transaction authorization id	
batchNumber	Optional	Batch number of the transaction	
dateProcessType	Optional	Add short filter. **Value / Short Key / Short Type** 0 / transactionId / DESC 1 / transactionId / ASC 2 / transmissionTime / ASC 3 / transmissionTime / DESC	
pan	Optional	SHA-256 value of the card number	
page	Optional	Page filter value of pagination	1
pageCount	Optional	Page count value of pagination	10 (0 is not

Parameter Name	Parameter Type	Description	Sample Value
			allowed)

6.2.2 Void List Response

VoidListResponse

```
[{"transactionId":"100007072027","authorizationId":null,"transmissionDate":200220,"transmissionTime":125326,"amount
```

Table 6.2.2 Void List Response Parameters

Parameter Name	Description	Sample Value
transactionId	Transaction unique id.	"100004892005"
authorizationId	Transaction authorization id	
transmissionDate	Transaction date	190925
transmissionTime	Transaction time	85238
amount	Amount value of the transaction	8 (0,08 €)
batchNumber	Batch number of the transaction	
maskedPan	Last 4 digits of PAN	2032

6.3 Refund List

In order to list refund transaction, sendCustomRequest method must be used. You must create your request and response classes. You can find the expected request and response for get void list request below.

6.2.1 Refund List Request

RefundListRequest

```
{"transactionId":"12345","startDate":20200205","endDate":20200219,"amount":1000,"amount":3000,"amountType":1,"autho
```

Table 6.2.1 Refund List Request Parameters

Parameter Name	Parameter Type	Description	Sample Value
transactionId	Optional	Transaction unique id	100004892014
startDate	Mandatory	start date to make query for transaction date	20190924
endDate	Mandatory	end date value to make query for transaction date	20191024
amount	Optional	Transaction amount value	100 for 1,00
secondAmount	Optional	Transaction amount value	100 for 1,00
amountType	Optional	Add amount query with below parameter. **Value / Type / Description** 0 / Empty / no filter is applied 1 / LessThan / less than given amount 2 / LessThanOrEqualTo / less than or equal togiven amount 3 / GreaterThan / greater than given amount 4 / GreaterThanOrEqualTo / greater than or equal to given amount	

<table>
<tr><th>Parameter Name</th><th>Parameter Type</th><th>Description</th><th>Sample Value</th></tr>
<tr><td></td><td></td><td><table><tr><th>Value</th><th>Type</th><th>Description</th></tr><tr><td>5</td><td>GreaterThanOrEqualTo & LessThanOrEqualTo</td><td>greater or equal to amount and less or equal to secondAmount</td></tr></table></td><td></td></tr>
<tr><td>batchNumber</td><td>Optional</td><td>Batch number of the transaction</td><td></td></tr>
<tr><td>dateProcessType</td><td>Optional</td><td>Add sort filter.<table><tr><th>Value</th><th>Short Key</th><th>Short Type</th></tr><tr><td>0</td><td>transactionId</td><td>DESC</td></tr><tr><td>1</td><td>transactionId</td><td>ASC</td></tr><tr><td>2</td><td>transmissionDate and transmissionTime</td><td>ASC</td></tr><tr><td>3</td><td>transmissionDate and transmissionTime</td><td>DESC</td></tr></table></td><td></td></tr>
<tr><td>authorizationId</td><td>Optional</td><td>Transaction authorization id</td><td></td></tr>
<tr><td>pan</td><td>Optional</td><td>SHA-256 value of the card number</td><td></td></tr>
<tr><td>page</td><td>Optional</td><td>Page value of pagination</td><td>1</td></tr>
<tr><td>pageCount</td><td>Optional</td><td>Page count value of pagination</td><td>10 (0 is not allowed)</td></tr>
</table>

6.3.2 Refund List Response

RefundListResponse

```
[{"transactionId":"100007072027","authorizationId":null,"transmissionDate":200220,"transmissionTime":125326,"amount
```

Table 6.3.2 Refund List Response Parameters

Parameter Name	Description	Sample Value
transactionId	Transaction unique id.	"100004892005"
authorizationId	Transaction authorization id	
transmissionDate	Transaction date	190925
transmissionTime	Transaction time	085238
amount	Amount value of the transaction	8 (0,08 €)
batchNumber	Batch number of the transaction	
maskedPan	Last 4 digits of PAN	2032

6.4 Report List

In order to list transaction report, sendCustomRequest method must be used. You must create your request and response classes. You can find the expected request and response for get report list request below.

6.4.1 Report List Request

ReportListRequest

```
{"startDate":20200205","endDate":20200219,"amount":1000,"secondAmount":3000,"amountType":1,"authorizationId":"12345
```


Table 6.4.1 Report List Request Parameters

Parameter Name	Parameter Type	Description	Sample Value
startDate	Mandatory	start date to make query for transaction date (yyyyMMdd)	20190924
endDate	Mandatory	end date value to make query for transaction date (yyyyMMdd)	20191024
amount	Optional	Transaction amount value	100 for 1.00, 1003 for 10.03
secondAmount	Optional	Transaction amount value	100 for 1,00, 1003 for 10.03
amountType	Optional	Add amount query with below parameter. **Value / Type / Description** 0 / Empty / no filter is applied 1 / LessThan / less than given amount 2 / LessThanOrEqualTo / less than or equal togiven amount 3 / GreaterThan / greater than given amount 4 / GreaterThanOrEqualTo / greater than or equal to given amount 5 / GreaterThanOrEqualTo & LessThanOrEqualTo / greater or equal to amount and less or equal to secondAmount	
dateProcessType	Optional	Add sort filter. **Value / Short Key / Short Type** 0 / transactionId / DESC 1 / transactionId / ASC 2 / transmissionDate and transmissionTime / ASC 3 / transmissionDate and transmissionTime / DESC	
authorizationId	Optional	Transaction authorization id	
approvalCodeType	Optional	Add approval code filter. **Value / Description** 0 / All transactions 1 / Success transactions 2 / Fail transactions	
page	Optional	Page value of pagination	1
pageCount	Optional	Page count value of pagination	10 (0 is not allowed)

6.4.2 Report List Response

ReportListResponse


```
[{"transactionId":"007216000049","pan":"C46BB8D8FD4D54D44534B4960F2C9FD84DB842C7841BB674B08410B45AFE2FD9","transmis
```


Table 6.4.2 Report List Response Parameters

Parameter Name	Description	Sample Value
transactionId	Transaction unique id.	"100004892005"
pan	SHA-256 value of the card number	
transmissionDate	Transaction date	190925
transmissionTime	Transaction time	085238
amount	Amount value of the transaction	8 for 0,08
approvalCode	Transaction result code	00
authorizationId	Transaction authorization id	213123
transactionType	Transaction type	0100 - SALE 0200 - VOID 0300 - REFUND 0400 - REVERSAL 0500 - VOID CANCEL 0600 - BATCH UPLOAD 0700 - SETTLEMENT 0800 - STATISTICS
batchNumber	Batch number of the transaction	17
stan	System Trace Audit Number	00000004
maskedPan	Last 4 digits of PAN	2032
kernelId	Card Brand Id	"31" - JCBAndVisa "02" - Mastercard "03" - Visa "04" - Amex "35" - JCB "06" - Troy "07" - UnionPay "0D" - Rupay "2D" - Pure

6.5 Send Custom Data To Host

In order to send a custom data to authorization service using transaction api, sendCustomRequest method must be used. You should create your response classes or get response as a json string.

sendCustomRequest

```
public <E,T> void sendCustomRequest(final RequestType requestType, final E request, final Class responseClassType,
```

sendCustomRequest

```
public <E,T> void sendCustomRequest(final RequestType requestType, final E request, final Class responseClassType,
```

Table 6.5.1 Host Custom Request Parameters

Parameter Name	Description	Sample Value
requestType	Type of request	`RequestType.SEND_TO_HOST`

request	It must be String or Map<String, String> format. The value must be Hex format.	For String Type : "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
		For Map<String,String> Type
		`Map<String, String> request = new HashMap<>();`
		`message.put("DE62", "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF");`

responseClassType	Class type of response object. It is required for deserialize response to given class type. Note: To get response as a json string. Set String.Class	HostCustomResponse.class
customRequestListener<T>	Asynchronous caller function. Note: To get response as a json string. Set CustomRequestListener<String>	
mti	Type of custom operation	0980 or 0990 Defualt MTI is 0980

6.5.2 Custom Host Response

CustomHostResponse

```
{
    "emvElements": [
        {
            "tag": "f37",
            "value": "313030303031393931303131",
            "fieldFormat": null
        },
        {
            "tag": "f39",
            "value": "3030",
            "fieldFormat": null
        }
    ]
}
```

Table 6.5.2 Custom Host Response Parameters

Parameter Name	Description	Sample Value
tag	Response tag field.	"f37"

Parameter Name	Description	Sample Value
	<table><tr><th>Value</th><th>Description</th></tr><tr><td>f37</td><td>Request Reference Number</td></tr><tr><td>f39</td><td>Response Code</td></tr></table>Note: If f39 value is not 3030 which means success, f37 tag may not exist.	
value	Hex format response value field	"313030303031393931303131"

Example 6.5.1 Host Custom Response Class

HostCustomResponse

```
public class HostCustomResponse {

    @Json(name = "emvElements")
    public List<EmvElement> emvElements;
}


public class EmvElement {

    @Json(name = "tag")
    public String tag;

    @Json(name = "value")
    public String value;
}
```

Example 6.5.2 Send Custom Data To Host Method Usage

SendCustomDataToHostMethodUsage

```
SoftPosApp.getSoftPosService().sendCustomRequest(RequestType.SEND_TO_HOST, "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF", Stri
    @Override
    public void onGetCustomResponse(final String response) {

        for(EmvElement emvElement : response.emvElements){
            Log.d("SEND_TO_HOST", emvElement.tag +" : " +emvElement.value);
        }
    }

    @Override
    public void onCustomRequestError(final SoftPosError error) {

    }
});
```

6.6 Transaction History

In order to list transaction history, sendCustomRequest method must be used. You must create your request and response classes. You can find the expected request and response for get transaction history list request below.

6.6.1 Transaction History Request

TransactionHistoryRequest

```
{"page":0,"pageCount":10,"transmissionDate":220627}
```

Table 6.6.1 Transaction History Request Class Parameters

Parameter Name	Parameter Type	Description	Sample Value
transmissionDate	Integer(Mandatory)	Transaction date(format = "yyMMdd ")	220925
transactionId	String	Transaction unique id	100004892014
mti	String	Type of custom operation	0980 or 0990 Defualt MTI is 0980
processingCode	String	Processing Code	020000
amount	Integer	Transaction amount value	100 for 1,00
dateProcessType	Integer	Date process type	
originalRrn	String	RRN is a unique identifier for the SP requests which is generated by SP	
startDate	Integer	start date to make query for transaction date	20190924
endDate	Integer	end date value to make query for transaction date	20191024
pageCount	Integer(Mandatory)	Record per page count	10
page	Integer(Mandatory)	Page	1

6.6.2 TransactionListResponse

TransactionHistoryResponse


```
{
  "transactions":[
  {"transactionId":"217809360220",
   "authorizationId":"",
   "acquirerId":,
   "terminalId":"",
   "deviceId":,"merchantId":"",
   "l3Mti":"0100",
   "l3ProcessingCode":"020000",
   "stan":"",
   "isoStan":null,
   "mti":"0200",
   "processingCode":"000000",
   "transmissionDate":220627,
   "transmissionTime":113046,
   "amount":1600,
   "approvalCode":"00",
   "maskedPan":"7687",
   "refNo":null,
   "groupNo":null,
   "aid":"A0000000041010",
   "label":"Debit Mastercard",
   "installmentCount":1,
   "originalRrn":null,
   "mcc":"0001",
   "kernelId":null,
   "approvedStatus":1}],
  "total":19,"pageCount":19
}
```

Table 6.6.2 TransactionListResponse Class Parameters

Parameter Name	Parameter Type	Description	Sample Value
transactions	TransactionRecord		
total	int		
pageCount	int		

Table 6.6.2 TransactionRecord Class Parameters

Parameter Name	Parameter Type	Description	Sample Value
transactionId	String	Transaction unique id	217809360220
approvalCode	String	Transaction result code	00
approvalStatus	String	Transaction status	1
authorizationId	String	Transaction authorization id	123456
transmissionDate	String	Transaction date	220627
transmissionTime	String	Transaction time	113046
amount	long	Transaction amount value	100 for 1,00
batchNumber	Integer	Batch number of the transaction	
maskedPan	String	Last 4 digits of PAN	2032
stan	String	System Trace Audit Number	000000004
isoStan	String	Iso standart	null
orderId	String	Order id	
mti	String	Transaction mti type - customer	0200
l3Mti	String	Transaction mti type	0100
approvedStatus	int	Approved Status	1

6.6.3 Method Usage

SendCustomDataToHostMethodUsage

```
TransactionHistoryRequest transactionHistoryRequest = new TransactionHistoryRequest();
transactionHistoryRequest.setPage(0);
transactionHistoryRequest.setRecordPerPage(10);
transactionHistoryRequest.setTransmissionDate("220627");

SoftPosApp.getSoftPosService().sendCustomRequest("getTransactionsV2", transactionHistoryRequest, TransactionListRes
    @Override
    public void onGetCustomResponse(TransactionListResponse transactionLists) {
        TransactionRecord[] records = transactionLists.getTransactions();
    }

    @Override
    public void onCustomRequestError(final SoftPosError error) {

    }
});
```

NOT : You can prepare a detail page by using the TransactionRecord list to view the transaction details.

7. Receipt Data

7.1 Get Receipt Data

This method is used to get receipt data. You can find the related information about transaction for receipt.

getReceiptData

```
public void getReceiptData(final String transactionId, final TransactionType transactionType, final ReceiptDataList
```

Table 7.1.1 Get Receipt Data Parameters

Parameter Name	Description	Sample Value
transactionId	Transaction unique id	929616000304
transactionType	Type of the transaction for getting related receipt data	TransactionType.SALE TransactionType.VOID TransactionType.REFUND
receiptDataListener	Asynchronous caller function.	

7.1.1 Receipt Data Listener

The response of the getReceiptData method can be reached using ReceiptDataListener methods. An exemplary usage is below:

VoidListListener

```
public interface ReceiptDataListener {

    void onGetReceiptData(HashMap<String, String> receiptData);

    void onGetReceiptDataError(SoftPosError error);
}
```

Table 7.1.2 Void List Listener Methods

Method Name	Description	Action
onGetReceiptData	Receipt data is successfully gotten.	Get the related information in response map object and prepare the receipt
onGetReceiptDataError	When SDK encounters unhandled error during getReceiptData call, this method will be triggered with the error message.	Get error detail using SoftPosError class. Please check 8. SoftPos Error section.

7.1.2 Receipt Data Result

You can reach receipt data tag in map object. You can find SoftPos and bank keys details for receipt data in tables below. SoftPos keys are constant but banks keys can be change. Also you you can find the sample usage below.

Table 7.1.3 Receipt Data Result SoftPos Key

Key	Description	Sample value
CT100	Last 4 digits of PAN	7733
CT101	Batch Number	67
CT102	AID	A0000000041010
CT103	Original Transaction Reference Number	929616000304
CT104	Merchant Info	Vakıf Katılım Merchant Arı: 3

<table>
<tr><th>Key</th><th>Description</th><th>Sample value</th></tr>
<tr><td>CT105</td><td>Transaction type

P.S : You must set the return type of CT105 via ACQUIRER_TRANSACTION_LANG parameter of Acquirer Parameter section on SoftPos Management Portal. The default value is enumarator.</td><td><table>
<tr><th>Enumerator</th><th>English</th></tr>
<tr><td>0100</td><td>SALE</td></tr>
<tr><td>0200</td><td>VOID</td></tr>
<tr><td>0300</td><td>REFUND</td></tr>
<tr><td>0400</td><td>REVERSAL</td></tr>
<tr><td>0500</td><td>VOID CANCEL</td></tr>
<tr><td>0600</td><td>BATCH UPLOAD</td></tr>
<tr><td>0700</td><td>SETTLEMENT</td></tr>
<tr><td>0800</td><td>STATISTICS</td></tr>
</table></td></tr>
<tr><td>CT106</td><td>Transaction date (dd/MM/yyyy) (Mobile Device)</td><td>23/10/2019</td></tr>
<tr><td>CT107</td><td>Transaction time (HH:mm) (Mobile Device)</td><td>12:34</td></tr>
<tr><td>CT108</td><td>Card Type</td><td>JCBAndVisa
Mastercard
Visa
Amex
JCB
Troy
UnionPay
Rupay
Pure
Unknown</td></tr>
<tr><td>CT109</td><td>Application Label</td><td>Mastercard
Maestro
Visa</td></tr>
<tr><td>CT110</td><td>Transaction timestamp (RFC 3339 time format) (generated by Backend)</td><td>2020-08-12T3:14:46+0300</td></tr>
<tr><td>CT111</td><td>CVM Type</td><td>"2" Online PIN
"1" CD-CVM is performed
"0" No-CVM</td></tr>
<tr><td>CT112</td><td>Masked PAN</td><td>**********4378</td></tr>
<tr><td>CT113</td><td>Application preferred name</td><td>CREDITO DE VISA</td></tr>
<tr><td>CT114</td><td>Acquirer name</td><td>P4Cards</td></tr>
<tr><td>CT115</td><td>Card Reading Method</td><td>Contactless</td></tr>
<tr><td>CT116</td><td>AIIC</td><td>Acquirer Id</td></tr>
<tr><td>CT117</td><td>Processing Type</td><td>Online</td></tr>
<tr><td>CT118</td><td>Currency code</td><td>972</td></tr>
<tr><td>CT119</td><td>Courtesy message</td><td>Thank you and good bye</td></tr>
</table>

Table 7.1.4 Receipt Data Result Host Key

Key	Description	Sample value
f04	Amount	000000000126

Key	Description	Sample value
f11	Stan	000306
f12	Transaction Time	163535
f13	Transaction Date (MMdd)	1023
f30	Original Amount	000000000000
f37	Transaction ID	100005041017
f38	Authorization Number	304163
f39	Authorization Response Code	3 digits or 2 digits
f41	POS Terminal Number	00192136
f42	Merchant Id	000000009041042
f43	Merchant Name and Location	

ParseReceiptData

```
String maskedPan = receiptData.get("CT100");

String amount = receiptData.get("f04");
```

7.2 Send Receipt Data With Email/SMS

This method is used to send receipt data by mail/sms.

sendReceiptData

```
sendCustomRequest(final String path, final E request, final Class responseClassType, final CustomRequestListener<T>
```

Table 7.2.1 Send Custom Request Data Parameters

Parameter Name	Description	Sample Value
`path`	`Named replacement in a URL path segment.`	`"sendReceiptEMAilAsync"`
`request`	Instance of request class. The class should have @Json annotation for related field. Note: To send raw json, set json string.	Check Example 7.2.1 Request Class
`responseClassType`	Class type of response object. It is required for deserialize response to given class type. Note: To get response as a json string. Set String.Class	ListResponse[].class
`customRequestListener`	Asynchronous caller function.	

Example 7.2.1 Request Class

Request Data

ListRequest

```
{"emvElements":[{"fieldFormat":"ANS","tag":"DF9997","value":"222825447185"},{"tag":"f133","value":"{\"displayLang\"
```

Request Class

```
public class EmailSmsRequest {
    @Json(name = "mti")
    private String mti;
```

```
    @Json(name = "emvElements")
    private List<EmvElement> emvElements = null;

    // you can add getter and setter methods if needed

    public static class F133 {

        @Json(name = "gsmNumber")
        private String gsmNumber;

        @Json(name = "countryCode")
        private String countryCode;

        @Json(name = "emailAddress")
        private String emailAddress;

        @Json(name = "receiptURL")
        private String receiptURL;

        @Json(name = "originalTransactionId")
        private String originalTransactionId;

        @Json(name = "transactionMti")
        private String transactionMti;

        @Json(name = "terminalId")
        private String terminalId;

        @Json(name = "displayLang")
        private String displayLang;


        @Json(name = "iso_stan")
        private String iso_stan;

        @Json(name = "receiptDeliveryType")
        private String receiptDeliveryType;

       //you can add getter and setter methods if needed
    }

    public static class EmvElement {

        @Json(name = "fieldFormat")
        private String fieldFormat;
        @Json(name = "tag")
        private String tag;
        @Json(name = "value")
        private String value;

        //you can add getter and setter methods if needed
    }
}
```

Example 7.2.2 Response Class

Response Data

Response

```
{"emvElements":[{"tag":"f39","value":"3030","fieldFormat":null}]}
```

Response Class

```
public class EmailSmsResponse {

    @Json(name = "emvElements")
    private List<EmailSmsRequest.EmvElement> emvElements = null;
```

```
    //you can add getter and setter methods if needed

    public static class F133 {

        @Json(name = "gsmNumber")
        private String gsmNumber;

        @Json(name = "originalTransactionId")
        private String originalTransactionId;

        @Json(name = "transactionMti")
        private String transactionMti;

        @Json(name = "terminalId")
        private String terminalId;

        //you can add getter and setter methods if needed

    }

    public static class EmvElement {

        @Json(name = "fieldFormat")
        private String fieldFormat;
        @Json(name = "tag")
        private String tag;
        @Json(name = "value")
        private String value;

       //you can add getter and setter methods if needed

    }
}
```

Example 7.2.3 Method Usage (EMail)

SendCustomDataToHostMethodUsage

```
EmailSmsRequest emailSmsRequest = new EmailSmsRequest();
emailSmsRequest.setMti("0620");
EmailSmsRequest.EmvElement emvElement = new EmailSmsRequest.EmvElement();
emvElement.setFieldFormat("ANS");
emvElement.setTag("DF9997");
emvElement.setValue(transactionId);

EmailSmsRequest.EmvElement emvElement2 = new EmailSmsRequest.EmvElement();
emvElement2.setFieldFormat(null);
emvElement2.setTag("f133");

EmailSmsRequest.F133 f133 = new EmailSmsRequest.F133();
f133.setEmailAddress(email);
f133.setDisplayLang(getLanguage());
f133.setTransactionMti(transactionType); // 0100, 0200, 0300
f133.setTerminalId(terminalId);
f133.setReceiptURL(receiptURL);
f133.setReceiptDeliveryType("EMAIL");


if (transactionType) == Transaction.REFUND) //300
    f133.setOriginalTransactionId(getTransactionId());

emvElement2.setValue(jsonAdapter.toJson(f133));

List<EmailSmsRequest.EmvElement> emvElementslist = new ArrayList<EmailSmsRequest.EmvElement>();
emvElementslist.add(emvElement);
emvElementslist.add(emvElement2);
```

```
emailSmsRequest.setEmvElements(emvElementslist);
SoftPosApp.getSoftPosService().sendCustomRequest("sendReceiptEMAilAsync", emailSmsRequest, EmailSmsResponse.class ,
    @Override
    public void onGetCustomResponse(EmailSmsResponse smsResponse) {

        for (int i = 0; i < smsResponse.getEmvElements().size(); i++) {
            if (smsResponse.getEmvElements().get(i).getTag().equalsIgnoreCase("f39") &&
                    smsResponse.getEmvElements().get(i).getValue().equalsIgnoreCase("3030")) {
                //SUCCESS
                break;
            }
        }
    }

    @Override
    public void onCustomRequestError(SoftPosError softPosError) {
        //FAIL
    }
});
```

Example 7.2.4 Method Usage (SMS)

SendCustomDataToHostMethodUsage

```
EmailSmsRequest emailSmsRequest = new EmailSmsRequest();
emailSmsRequest.setMti("0620");
EmailSmsRequest.EmvElement emvElement = new EmailSmsRequest.EmvElement();
emvElement.setFieldFormat("ANS");
emvElement.setTag("DF9997");
emvElement.setValue(transactionId);


EmailSmsRequest.EmvElement emvElement2 = new EmailSmsRequest.EmvElement();
emvElement2.setFieldFormat(null);
emvElement2.setTag("f133");


EmailSmsRequest.F133 f133 = new EmailSmsRequest.F133();
f133.setGsmNumber(getNumber());
f133.setCountryCode(String.valueOf(smsET.getSelectedCountry().getDialCode()));
f133.setTransactionMti(transactionType);  // 0100, 0200, 0300
f133.setDisplayLang(getLanguage());
f133.setTerminalId(terminalId);
f133.setReceiptURL(receiptURL);
f133.setIso_stan(isoStan);
f133.setReceiptDeliveryType("SMS");


if (transactionType) == Transaction.REFUND)
    f133.setOriginalTransactionId(getTransactionId());


emvElement2.setValue(jsonAdapter.toJson(f133));

List<EmailSmsRequest.EmvElement> emvElementslist = new ArrayList<EmailSmsRequest.EmvElement>();
emvElementslist.add(emvElement);
emvElementslist.add(emvElement2);

emailSmsRequest.setEmvElements(emvElementslist);


SoftPosApp.getSoftPosService().sendCustomRequest("sendReceiptEMAilAsync", emailSmsRequest, EmailSmsResponse.class ,
    @Override
    public void onGetCustomResponse(EmailSmsResponse smsResponse) {

        for (int i = 0; i < smsResponse.getEmvElements().size(); i++) {
            if (smsResponse.getEmvElements().get(i).getTag().equalsIgnoreCase("f39") &&
```

```
                    smsResponse.getEmvElements().get(i).getValue().equalsIgnoreCase("3030")) {
                //SUCCESS
                break;
            }
        }
    }

    @Override
    public void onCustomRequestError(SoftPosError softPosError) {
        //FAIL
    }
});
```

8. SoftPos Error

8.1 Description

The error messages of the SoftPOS SDK is type of SoftPosError class. Error codes or error messages of SoftPosError instance should not be displayed directly, rather they should be constructed as error messages to be displayed for the consumers.

TransactionListener

```
public class SoftPosError {

    public int getErrorCode();

    public String getErrorMessage();

    public int getInternalErrorCode();

    public int getInternalErrorSubCode();

    public String getInternalErrorMessage();
}
```

Table 8.1.1 Transaction Listener Methods

Method Name	Description	Sample Value
getErrorCode	It is used to get error code.	104
getErrorMessage	It is used to get error message.	Server Error
getInternalErrorCode	It is used to get error code received from SoftPOS server.	2
getInternalErrorSubCode	It is used to get error sub code received from SoftPOS server.	1
getInternalErrorMessage	It is used to get error message received from SoftPOS server or SDK exception message.	Session is not valid

Table 8.1.2 Error Codes and Messages in SoftPos SDK

Error Code	Error Message	API	Description	Possible Action
100	Internal Error	• All APIs	SDK encounters unhandled exceptions	Show an error message to user like "Currently your operation cannot proceed" Redirect user main page.
101	Connection Error	• initialize • register • checkPOSService • getReceiptData • sendCustomRequest	SDK encounters a connection error or timeout.	Show an error message to user like "Service not available. Please try again later" Check device has an Internet connection.
102	Location could not be found	• register • checkPOSService • getReceiptData • sendCustomRequest	SDK cannot get current location.	Ask user to enable location service and change location method to High accuracy or power saving mode.
103	SafetyNet Api Error	• initialize • register • checkPOSService • getReceiptData • sendCustomRequest	Safetynet Api returns an unknown error.	Show an error message to user "Service is not available. Please try again later" Contact your system admin.

Error Code	Error Message	API	Description	Possible Action
104	Server Error	• initialize • register • checkPOSService • getReceiptData • sendCustomRequest	SoftPos backend returns an error.	Show an error message to user like "Service is not available. Please try again later" Check internalErrorMessage for error details.
105	CheckPosService is not successful	• startTransaction	A success response from checkPOSService method call is prerequisite for calling startTransaction method.	Call checkPOSService method before calling startTransaction method.
106	Kernel Init Error	• initialize	SDK cannot initialized I2 kernel.	Show an error message to user like "Currently your operation cannot proceed" Pos parameters may be invalid. Contact your system admin.
107	SDK Error Response Message	• initialize	Device keys are empty	Mobile Key version is empty
108	SDK Error Response Message	• startTransaction	Not ready to Transaction	When Transaction started and failed
109	Another transaction in progress	• startTransaction	SDK cannot start another transaction if it is already a started once.	In order to start new transaction, you must cancel current one or wait until transaction finish or timeout.
110	Device does not have NFC reader	• initialize	SDK cannot detect any NFC reader on device. SDK cannot be used in device.	Show an error message to user like "Your device does not support SoftPOS because of NFC capability."
111	Kernel Init Error	• initialize	Parameter set is invalid	Parameter set is invalid, Please Contact Customer Support
112	Local Data could not clear	• unregister	SDK encounters unhandled exceptions during local data removal.	Show an error message to user like "Currently your operation cannot proceed"
113	SDK Error Response Message	• initialize	TMK is empty	Terminal Master Key is not generated
114	SDK Error Response Message	• initialize	MTEK is empty	Terminal MTEK Key is missing
115	SDK Error Response Message	• initialize	MTAK is empty	Terminal MTAK Key is missing
116	SDK Error Responce Message	• initialize	MTPK is empty	Terminal MTPK Key is missing
117	SDK Error Response Message	• initialize	MTSK is empty	Terminal MTSK Key is missing
118	SDK Error Response Message	• initialize	Encryption is fail	!NOT USED

Error Code	Error Message	API	Description	Possible Action
119	Google Play Service is not available	• initialize • register • checkPOSService • getReceiptData • sendCustomRequest	SDK cannot find google play services for safetynet support.	Show an error message to user like "Your device does not support SoftPOS because Google Play Service is not available" Contact your system admin.
120	SDK is not initialized	• register • checkPOSService • getReceiptData • sendCustomRequest	initialize method is not be called before current operation.	initialize must be first call from SDK. Please be sure calling initialize before calling other methods.
121	SDK Error Response Message	• initialize	PIN RKI Error	Remote Key Injection has encountered a problem. Check application permissions.
122	SDK Error Response Message	• initialize	PIN APP Error	Pin App closed due to the timeout or user cancellation
123	SDK Error Response Message	• initialize	PIN RKI neded	Pin App needs to download Pin Keys.
124	SDK Error Response Message	• initialize	Pin App needs extra permission	Pin app extra permissions are not granted.
125	SDK Error	• register	Pin App cannot be found	Pin App is not installed
126	Already registered SDK.	• register	SDK is already completed registration process.	If you want to register again, please call unregister method first then you can call register method.
127	NFC is not active.	• startTransaction	NFC is not enabled on the device.	Show an error message to user to enable NFC on the device
128	Request type is not defined	• sendCustomRequest	Request parameter type is not supported on SDK.	Check the integration document for supported parameter types.
129	Device keys are damaged	• initialize • checkPOSService	SDK cannot revert device keys because of os update or security patch.	Call register method.
130	SDK Error Response Message	• initialize	No service/api found in requested path	Service API path is wrong or not reachable.
131	SDK Error Response Message	• startTransaction	Kernel Parameters are not verified	Integration data is not correct, amount or MTI is missing
132	SDK Error Response Message	• startTransaction	Required transaction listener is not provided	Transaction Listener Error
133	SDK Error Response Message	• initialize	SDK Required backend minimum version does not match with host	Core version does not match with SDK. Core version upgrade is required.

9. Pin Integration

SoftPos SDK support pin entering for over CVM limit transaction with Pin Application. This section is described how to enable Pin Application on SoftPos SDK.

9.1 Sign Pin Application

Your SoftPos application and Pin Application must be signed with the same certificate. You can use apksigner tool to sign pin application. The sample usage is below.

apksigner

```
apksigner sign --ks YOUR_CERTIFICATE.jks --ks-pass 'pass:YOUR_PWD' --out SoftPosPinApp-releasse.apk SoftPosPinApp-u
```

9.2 Import Pin SDK

SoftPosPIN SDK is dependency of SoftPos SDK for Pin Transaction. You have to add SoftPosPIN SDK on your project like below. You do not need to make any integration with SoftPosPIN SDK. SoftPos SDK is already integrated with SoftPOS PIN SDK.

Dependencies

```
implementation(name:'SoftPosPIN-v1.0.0', ext:'aar')
```

9.3 Permissions

SoftPosPIN SDK needs some permissions using in lifecycle. Permissions are listed below. MessengerService permission is not needed to be added in your project, because SDK manifest file already has this permission. If you enable auto kill for Pin Application, you have to add "KILL_BACKGROUND_PROCESSES" permission on your manifest file.

Permissions

```
<uses-permission android:name="com.permission.messengerservice" />

<uses-permission android:name="android.permission.KILL_BACKGROUND_PROCESSES" />
```

9.4 SoftPosInfo Class

Set Pin Application related parameters to SoftPos SDK using SoftPosInfo Class

Table 9.4.1 SoftPos Info Parameter

Parameter Name	Description	Default Value
PinAppLanguage	Language of error and info messages on PIN App.	en
PinAppTimeoutMs	Timeout value for entering pin on PIN App. If you change this parameter do not forget to update L2HostResponseTimeoutMs because SoftPos SDK waits online response until L2HostResponseTimeoutMs value.	60000 ms
PinAppPackageName	SoftPos SDK can communicate PIN App with pin app package name value.	com.paycore.softpos.pin
PinAppAutoKill	SoftPos SDK can automatically kill pin application when pin application goes background. If you enable this feature you have to add	false

Parameter Name	Description	Default Value
	"KILL_BACKGROUND_PROCESSES" permission on your manifest file.	

An exemplary usage is below:

Setting Program Parameters

```
SoftPosInfo.setPinAppLanguage("tr");

SoftPosInfo.setPinAppTimeoutMs(90000);

SoftPosInfo.setPinAppPackageName("com.example.pin");

SoftPosInfo.disablePinAppAutoKill();

SoftPosInfo.enablePinAppAutoKill();
```

9.5 Permissions

SoftPOS SDK needs some permissions for Pin Integration. Permissions are listed below.

Permissions

```
<uses-permission android:name="android.permission.KILL_BACKGROUND_PROCESSES" />
```

9.6 Pin App Listener

PinAppListener methods are triggered for over CVM transactions. You can use these method for your UI animations. Set your listener via setPinAppListener method of the SDK.

setPinAppListener

```
public void setPinAppListener(PinAppListener pinAppListener)
```

TransactionListener

```
public interface PinAppListener {

    void onPinAppStarting();

    void onPinAppClosed();
}
```

Table 9.6.1 Pin App Listener Methods

Method Name	Description	Action
onPinAppStarting	It is triggered when SDK tries to open Pin App for over CVM transactions.	Show an indicator to user. Pin App is opening in a short time.
onPinAppClosed	It is triggered when Pin App is closed	Hide the indicator. SDK will triggers goOnline callback after preparing online data with the pin info.

9.7 Transaction Result

When transaction is completed, onCompleted method will be triggered. Result of the transaction can be received by using TransactionResult methods. In this section only pin related methods are described. Check `5 Start Transaction` section to more details.

TransactionResult

```
public class TransactionResult {

    public boolean isWithPin()

    public PinResult getPinResult()
}
```

Table 9.7.1 Transaction Result Methods

Parameter Name	Description	Sample Value
isWithPin getPinResult	Transaction is required pin entering or not. If transaction i is required pin entering, check pinResult object to more details.	true or false

9.7.1 Pin Result

When transaction is required pin entering, you can get the details about pin support transaction on pinResult object.

PinResult

```
public class PinResult {

    public boolean isPinReceived();

    public SoftPosError getPinError();

    public boolean isPinAppTimeoutOrCancelled();
}
```

Table 9.7.1.1 Pin Result Methods

Parameter Name	Description	Sample Value
isPinReceived getPinError	Returns pin value can successfully get via pin application or not. If an error occurs during pin transaction, you can get error details using this method.	true of false
isPinAppTimeoutOrCancelled	If pin application is cancelled or it is closed because of timeout value, this methods returns true.	false

9.7.2 Transaction Result Flow

9.8 SoftPos Error

PIN related errors are listed below.

Table 9.8.1 Error Codes and Messages in SoftPos SDK for Pin Integration

Error Code	Error Message	API	Description	Possible Action
121	Pin App RKI error	• register • checkPOSService	SDK cannot complete pin RKI process.	Show an error message to user like "Currently your operation cannot proceed" Contact your system admin.
122	Pin App unknown error	• startTransaction	Pin App returns an error.	Check error details.
124	Pin App permission error	• register	Pin App returns a permission error. It is only valid Xiaomi devices.	Grant background permission for Pin App.
125	PIN APP cannot be found	• register • transactionResult	SDK cannot found pin app packege name in device installed app list.	Check Pin App is installed on device or set correct pi app package name on SoftPosInfo.setPinAppPackageName("PIN_APP_PACKAGE_NAME method.

10. SDK Flow Diagram

10.1 Flow Diagram

The flow diagram of the main methods of SoftPOS SDK is shown below.

The flow of the application should start with Initialize.

Merchant application flow in case the fingerprint of the device is changed:

1. Merchant app calls one of SDK APIs
2. If SDK returns ErrorCode:104 & internalErrorCode:9 & internalErrorSubCode: 13, merchant application should invoke unregister api of SDK
3. Unregister response:
    a. If Merchant App gets onUnregisterSuccess callback, right after that app should use the stored informations(merchnatid, terminal id, activation code) to invoke register
    b. If Merchant App gets onUnregisterError callback, merchant app should show an error message to merchant.

This is the flow which should be implemented by merchant app in case fingerprint is changed on the device.

11. SoftPOS App Dependency List

Review Process of external libraries is explained in DevOps Process documentation. Each versions are checked whether they are up-to-date or not.

Vulnerability analysis of those dependencies is explained in DevOps Process documentation. SoftPOS Application is scanned by OWASP Dependency Check utility.

```
SoftPOS Application
==========================================
/** Retrofit */
implementation 'com.squareup.retrofit2:converter-moshi:2.9.0'
implementation 'com.squareup.okhttp3:logging-interceptor:4.7.2'
implementation 'com.squareup.retrofit2:converter-scalars:2.9.0'



/** Room components */
implementation 'android.arch.persistence.room:runtime:1.1.1'
annotationProcessor "android.arch.persistence.room:compiler:1.1.1"
androidTestImplementation 'android.arch.persistence.room:testing:1.1.1'


/** Google SafetyNet Service */
implementation "com.google.android.gms:play-services-safetynet:17.0.0"
 fullImplementation "com.google.android.gms:play-services-ads-identifier:18.0.1"

/** Huawei SafetyDetect Service*/
implementation 'com.huawei.hms:safetydetect:5.0.5.302'


/** Glide */
implementation 'com.github.bumptech.glide:glide:4.11.0'
annotationProcessor 'com.github.bumptech.glide:compiler:4.11.0'


/** Sun Mail*/
implementation 'com.sun.mail:android-mail:1.6.5'
implementation 'com.sun.mail:android-activation:1.6.5'


/** Progress Bar */
implementation (name:'kprogresshud-1.2.1', ext:'aar')


/** Zxing Library for QR Code */
implementation 'com.google.zxing:core:3.4.0'
implementation 'com.journeyapps:zxing-android-embedded:3.4.0@aar'


/** Circular ImageView */
implementation 'de.hdodenhof:circleimageview:3.1.0'


/** Toasty */
implementation 'com.github.GrenderG:Toasty:1.4.2'


/** ExoPlayer */
implementation 'com.google.android.exoplayer:exoplayer-core:2.11.7'
implementation 'com.google.android.exoplayer:exoplayer-dash:2.11.7'
implementation 'com.google.android.exoplayer:exoplayer-ui:2.11.7'

/** Visa Branding */
implementation (name:'VisaSensoryBranding', ext:'aar')

/** Internation Phone Input*/
implementation 'net.rimoto:intlphoneinput:1.0.1'
```

```
/** Flurry components */
fullImplementation 'com.flurry.android:analytics:14.0.0'
SoftPos SDK
==============================================
/** Mockito Test*/
testImplementation 'org.mockito:mockito-core:1.10.19'


/** Moshi */
coreImplementation "com.squareup.moshi:moshi:1.8.0"


/** Google SafetyNet */
fullImplementation "com.google.android.gms:play-services-safetynet:17.0.0"


/** Huawei SafetyDetect Service*/
implementation 'com.huawei.hms:safetydetect:5.0.5.302'


withKnoxForCoreCompileOnly files('src/withKnoxForCore/libs/knoxsdk.jar')


/** Retrofit */
fullImplementation 'com.squareup.retrofit2:converter-moshi:2.6.4'
fullImplementation 'com.squareup.retrofit2:converter-scalars:2.6.4'
fullImplementation 'com.squareup.okhttp3:okhttp-urlconnection:3.12.12'
fullImplementation 'com.squareup.okhttp3:logging-interceptor:3.9.1'


/** HTTP Logging */
fullImplementation'com.readystatesoftware.chuck:library:1.1.0'


/** Room components */
fullImplementation 'androidx.room:room-runtime:2.2.5'
fullAnnotationProcessor 'androidx.room:room-compiler:2.2.5'


/** Flurry components */
fullImplementation 'com.flurry.android:analytics:14.0.0'
```

3rd Library Name	Version	Used By	Used For
com.squareup.retrofit2	2.9.0	SoftPOS Application	REST abstraction of HTTP communication with backend
com.squareup.okhttp3	4.7.2	SoftPOS Application	Secure communication with backend
android.arch.persistence.room	1.1.1	SoftPOS Application	Database object mapping.
com.google.android.gms:play-services-safetynet	17.0.0	SoftPOS Application	Device health check with Safetynet API of Google
com.huawei.hms:safetydetect	5.0.5.302	SoftPOS Application	Device health check with SafetyDetect API of Huawei
com.github.bumptech.glide:glide	4.11.0	SoftPOS Application	Image Loading and caching
com.github.bumptech.glide:compiler	4.11.0	SoftPOS Application	Image Loading and caching
com.sun.mail:android-mail	1.6.5	SoftPOS Application	Sending e-mail
com.sun.mail:android-activation	1.6.5	SoftPOS Application	Sending e-mail

3rd Library Name	Version	Used By	Used For
kprogresshud	1.2.1	SoftPOS Application	Showing progress bar, for instance while registeration process.
com.google.zxing:core	3.4.0	SoftPOS Application	Creating QR Code for receipt.
de.hdodenhof:circleimageview	3.1.0	SoftPOS Application	Circulat profile image view
com.github.GrenderG:Toasty	1.4.2	SoftPOS Application	Showing toast messages to the user of SoftPOS Application.
`VisaSensoryBranding`	1.0.1	SoftPOS Application	Branding animations after a successful visa payment transaction
com.google.android.exoplayer:exoplayer	2.11.7	SoftPOS Application	Playing media.
net.rimoto:intlphoneinput	1.0.1	SoftPOS Application	Getting phone input internationally for sending SMS receipt.
org.mockito:mockito-core	1.10.9	SoftPOS SDK	JSON data handling for communication with SoftPOS Backend
com.squareup.moshi:moshi	1.8.0	SoftPOS SDK	JSON data handling for communication with SoftPOS Backend
com.google.android.gms:play-services-safetynet	17.0.0	SoftPOS SDK	Device health check with Safetynet API of Google
com.huawei.hms:safetydetect	5.0.5.302	SoftPOS SDK	Device health check with SafetyDetect API of Huawei
com.squareup.retrofit2	2.6.4	SoftPOS SDK	REST abstraction of HTTP communication with backend
com.squareup.okhttp3:okhttp-urlconnection	3.12.12	SoftPOS SDK	Secure communication with backend
com.squareup.okhttp3:logging-interceptor	3.9.1	SoftPOS SDK	Secure communication with backend
com.readystatesoftware.chuck:library	1.1.0	SoftPOS SDK	Inspecting HTTP request for debug purposes only.
androidx.room:room	2.2.5	SoftPOS SDK	Abstraction over SQLite library
com.flurry.android:analytics	14.0.0	SoftPOS SDK	Remote Loging library
com.google.android.gms:play-services-ads-identifier	18.0.1	SoftPOS SDK	`Ads identifier`

12. TestUI Integration

12.1 API Description

SoftPos SDK has Test UI integration and it is required for Kernel functional testing. SoftPos SDK and TestUI can communicate each other with tcp protocol. You should make an activity for TestUI which Laboratory can use it to make functional tests.

12.2 Permissions

SoftPOS SDK needs some permissions to communicate TestUI. Permissions are listed below, but they are not needed to be added in L3 UI/UX project because SDK manifest file already has these permissions.

Permissions

```
<uses-permission android:name="android.permission.INTERNET" />
```

12.3 Test UI IP and Port Configuration

SoftPosInfo class can use in order to set ip and port value to SoftPOS SDK.

Table 12.3.1 SoftPosInfo Methods

Method Name	Description	Default Value
SoftPosInfo.setTestUIIP	IP value of the TestUI application	192.168.137.1
SoftPosInfo.setTestUIPort	Port value of the TestUI application	5744

An exemplary usage is below:

Set L2 Host Response Timeout

```
SoftPosInfo.setTestUIIP("192.168.137.1");


SoftPosInfo.setTestUIPort(5744);
```

12.4 Start Test UI

The startTestUI method is used to start tcp connection to TestUI application. TestUI IP and Port value must be set before calling startTestUI method.

StartTestUI

```
public void startTestUI(Activity activity)
```

Table 12.4.1 Start Test UI Parameters

Parameter Name	Description
activity	Activity of the TestUI fragment. It is used to enable NFC for this activity.

12.5 Stop Test UI

The startTestUI method is used to stop tcp connection to TestUI application.

StopTestUI

```
public void stopTestUI()
```

12.6 Set Kernel Message Listener

During the kernel test, kernel send some messages for Test User. You should display these message in the activity. In order to get kernel message, you should register your callback function using setKernelMessageListener.

SetKernelMessageListener

```
public void setKernelMessageListener(KernelMessageListener kernelMessageListener)
```

Table 12.6.1 Set Kernel Message Listener Parameters

Parameter Name	Description
kernelMessageListener	Asynchronous caller function.

12.7 Kernel Message Listener

The Kernel messages can be reached using KernelMessageListener methods. An exemplary usage is below:

KernelMessageListener

```
public interface KernelMessageListener {

    void onMessageReceived(String message, int ms);
}
```

Table 12.7.1 Kernel Message Listener Methods

Method Name	Description	Action
onMessageReceived	It is triggered when Kernel send a message.	Display message o the screen. It is required for functional testing

13. Knox

13.1 APIs Description

SoftPOS SDK can use KNOX to reach device feature and restrict them during transaction. The list of the device features are below.

- Audio Recording
- Developer Mode
- Screen Capture
- Status Bar Expansion
- Video Recording
- USB Tethering
- Incoming SMS & MMS
- Outgoing SMS & MMS
- Emergency Call Only
- Change The State of GPS
- Notification

Warning : You must be have Device Admin authority and activate KPE license to use Knox feature restriction.

EnableKnoxFeature

```
public void enableKnoxFeatureRestriction()
```

The enableKnoxFeature method is used to enable Knox restriction. It is recommended calling this function with onStart method of fragment or activity which SoftPOS SDK is initialized.

DisableKnoxFeature

```
public void disableKnoxFeatureRestriction()
```

The disableKnoxFeature method is used to disable Knox restriction. It is recommended calling this function with onStop method of fragment or activity which SoftPOS SDK is initialized.

13.2 Permissions

SoftPOS SDK needs Knox permissions for using Knox feature. Permissions are listed below, but they are not needed to be added in L3 UI/UX project because SDK manifest file already has these permissions.

Permissions

```
<uses-permission android:name="com.samsung.android.knox.permission.KNOX_HW_CONTROL" />
<uses-permission android:name="com.samsung.android.knox.permission.KNOX_LOCATION" />
<uses-permission android:name="com.samsung.android.knox.permission.KNOX_PHONE_RESTRICTION" />
<uses-permission android:name="com.samsung.android.knox.permission.KNOX_RESTRICTION_MGMT" />
<uses-permission android:name="com.samsung.android.knox.permission.KNOX_APP_MGMT" />
```

14. Security Countermeasure

14.1 Check Dex Files Checksum

SoftPOS SDK needs some security checks on your project's dex files. SoftPOSFinalizeTool tool calculates your dex files checksum and writes circle-red.png image with our algorithm. Then on during runtime, our SDK will read this image then check dex files checksums and image's value. Instant Run must be disabled to calculate correct checksum value.

Tool Requirements

- Instant Run must be disabled
- Gradle Plugin version 3.3.0 and upper.

where options include:

Options	Sample	Description
-ip	circle-red-real.png	can be any image in your project.
-i	build/intermediates/transforms/dexMerger/0/classes.dex, build/intermediates/transforms/dexMerger/1/classes.dex	Dex Files array (gradle script will help)
-o	"$projectDir/src/main/assets/circle-red.png"	should

You can find the gradle task below that should be added as your project build gradle file. You can find the tool here SoftPOSFinalizeTool.zip and sample softpos.sdk.sample.rar

Android build.gradle Sample

```
def createTasks(com.android.build.gradle.api.ApplicationVariant variant) {

    String suffix = variant.name.capitalize()
    String prepareChecksum = "prepareSoftPosFinalizeTool" + suffix

    Task taskEncrypt = tasks.create(prepareChecksum) {

        doFirst {

            String inDexFiles = ''

            fileTree(dir: "build/intermediates/dex/" + variant.name,
                    includes: ['**/*.dex']).files.each {
                inDexFiles += it.getAbsolutePath()
                inDexFiles += ';'
            }

            exec {
                if (System.properties['os.name'].toLowerCase().contains('windows')) {
                    executable 'cmd'
                    args '/c', "java", '-jar', 'SoftPOSFinalizeTool.jar',
                            '-ip', "$projectDir/src/main/res/mipmap-hdpi/circle-red.png",
                            '-i', inDexFiles, '-o', "$projectDir/build/intermediates/merged_assets/" + variant.name
                } else {
                    commandLine "java", '-jar', 'SoftPOSFinalizeTool.jar',
                            '-ip', "$projectDir/src/main/res/mipmap-hdpi/circle-red.png",
                            '-i', inDexFiles, '-o', "$projectDir/build/intermediates/merged_assets/" + variant.name
                }
            }
        }
    }

    Task ndktask = tasks["merge"+suffix+"JniLibFolders"]
    ndktask.dependsOn(taskEncrypt)
}

android {

    def ENABLE_SOFTPOS_FINALIZE_JAR = "TRUE" // "TRUE" !!! Instant Run must be disabled !!!
```

```
    applicationVariants.all { variant ->
        if (ENABLE_SOFTPOS_FINALIZE_JAR == "TRUE") {
            // println variant.name
            createTasks(variant)
        }

    }
}
```

Caution 1: Please note that the output name is circle-red.png and the directory on assets. ('-o','$projectDir/build/intermediates/merged_assets/debug/mergeDebugAssets/out/circle-red.png")

14.2 Tamper Guard API

If SDK can detect any security issue, it takes action to protect own data and triggers "onRiskDetected" method to inform the integrator application that security risk is detected.

This listener should be set via SoftPosInfo class setDetectionListener method.

DetectionListerner

```
public interface DetectionListerner {

    void onRiskDetected();
}
```

14.3 Wipe

This method is used to wipe data in SDK. If application can detect any security problem in attestation check, it should "wipe" method to clear sensitive data in SoftPOS SDK.

Wipe

```
public void wipe()
```

14.4 Visa SDK Integration Security Requirement Table

Table 2–5: Client Application Requirements when Integrating an Approved SDK

	Client Application Requirements when using an Approved SDK
1.	The Tap to Phone solution must have a User Guidance document defining how to use the mobile acceptance client application in a secure manner, including secure installation, update, version control, and default settings.
2.	The mobile acceptance client application must only be installed and updated through either the official device Operating System (OS) or the Original Manufacturer Equipment (OEM) stores.
3.	Mobile acceptance client application must only use an SDK with an approved version(s) by Visa.
4.	Mobile acceptance client application must integrate an approved SDK in accordance with the SDK Integration Guidance document and not bypass, re-implement or override exposed API and security functions.

14.5 Security Guidance Document

This document is delivered with "YAZARA MPoC Software Security Guidance" document. Customers should follow the guidance items while integrating the MPOC SDK in a secure way. "YAZARA MPoC Software Security Guidance" document is distributed via email to the customers. Please contact info_softpos@yazara.com to get "YAZARA MPoC Software Guidance" document.

15. SoftPOS Runtime Error List

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
1	AcquirerHostConnectionException	7	14	Connection attempt is failed to acquirer host.	Can not connect to acquirer host	No	application.properties side IP port should be checked by Sys Admin from Acq Host	
2	AppPermissionException	10	26	App permissions are missing.		Yes	User will redirect to Risk Management → APK Whitelist → Android Permission column related permission should be given.	https://softpostest.paycore.com/softpos/#/apkwhiteli
3	AppPackageNameException	10	25	App package name is not in whitelist.		Yes	User will redirect to Risk Management → APK Whitelist → Application PackageName should be given.	https://softpostest.paycore.com/softpos/#/apkwhiteli
4	AcquirerNotFoundException	7	2	Acquirer is not found in system.	*	No	Acq should be defined in the system.	
5	AcquirerParameterDeserializeException	7	4	Cache error. Cache data is not valid in system.	*	Yes	User will redirect to Operation → Acq Flow Config → InOut Rule Config definitions should be checked.	https://softpostest.paycore.com/softpos/#/spsacqflo
6	AcquirerParameterMissingException	7	8	Definition error. Configuration data is not found in system.	*	Yes	User will redirect to Operation → Acq Parameter → missing key name should be defined.	https://softpostest.paycore.com/softpos/#/spsacqpar
7	AcquirerTerminalBusinessParameterMissingException	7	8	Definition error. Configuration data is not found in the system.	*	Yes	User will redirect to Operation → Terminal Management → TrmGroupID should be selected.	https://softpostest.paycore.com/softpos/#/spstermin

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
8	ActivationCodeNotMatchException	1	3	Activation is not match.	*	Yes	User will redirect to Operation → Terminal Management → Activation code should be entered.	https://softpostest.paycore.com/softpos/#/spsterminı
9	AttestationCheckFailedException	10	18	Attestation data is not valid in system.	"Cpu usage exceeds the limit defined by acquirer."	No	If this error occurs in the Prod environment, the terminal application will be at risk!	
10	AttestationDataFormatException	10	19	Attestation data format is not valid in system.	"Signature format error."	No	If this error occurs in the Prod environment, the terminal application will be at risk!	
11	AttestationTxnPaymentAmountLimitException	10	15	Transaction amount limit is exceed for terminal.	*	Yes	User will redirect to Risk Management→ Attestation Limit → amounts will be checked In the test environment, the transaction amount limit will be checked.	https://softpostest.paycore.com/softpos/#/limits/attes
12	AttestationTxnPaymentCountLimitException	10	16	Transaction count limit is exceed for terminal.	*	Yes	User will redirect to Risk Management→ Attestation Limit → Transaction count will be checked In the test environment, the transaction amount limit will be checked.	https://softpostest.paycore.com/softpos/#/limits/attes

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
13	AttestationTxnTimeLimitException	10	17	Temporary error. Time limit block count for device fingerprint is exceed with in range seconds temporary block will apply for seconds.	*	Yes	User will redirect to Risk Management→ Attestation Limit → Time limit will be checked In the test environment, the transaction amount limit will be checked.	https://softpostest.paycore.com/softpos/#/limits/attes
14	AttestationTxnTimeOverflowException	10	17	Transaction time is exceed the limits of overflow.	*	Yes	User will redirect to Risk Management→ Attestation Limit → Time Overflow Limit will be checked In the test environment, the transaction amount limit will be checked.	https://softpostest.paycore.com/softpos/#/limits/attes
15	AuthenticationDataException	1	4	Authentication data is not valid in system.	"Authorization data decode error."	No	Sys Admin should inspect the log	
16	AuthenticationException	1	9	Authentication data is not valid in system.	"Device id not match."	No	Sys Admin should inspect the log	
17	BatchUploadNeededException	3	44	Batch upload is needed.		Yes	User will redirect to Operation → Batch Monitoring → will click Settlement button acq side will be sent settlement message. An automatic Batch upload will be sent to the system.	https://softpostest.paycore.com/softpos/#/batchMon
18	DatabaseConnectionException	8	14	Database connection attemp is failed		No	DB Admin should check the definitions.	
19	CheckAppVersionException	10	21	App version is not match.		Yes	1- Force to update the application on the store 2- User will redirect to Risk Management → Apk Whitelist → Add New Apk Whitelist screen and a new Apk will be added.	https://softpostest.paycore.com/softpos/#/apkwhiteli

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
20	DeviceBlockedException	9	11	Device is blocked state.	*	No	The Device is at risk! Our guard fetches hook, root, emulator or debug detection. This device cannot be used again. A new registration protocol should be applied.	
21	DeviceCheckBlockedException	9	11	Device is blocked state.		No	The Device is at risk! Our guard fetches hook, root, emulator or debug detection. This device cannot be used again. A new registration protocol should be applied.	
22	DeviceKeysExpiredException	11	7	Device keys have expired.	"Device key rotation required."	No	SDK fixes that problem itself on the terminal. No action is needed.	
23	DeviceNotFoundException	9	2	Device is not found in system.	*	Yes	User will redirect to Operation → Device Management → will check the device list.	https://softpostest.paycore.com/softpos/#/devicemn
24	DeviceSwapedException	9	22	Device is blocked state because of swap.	*	No	The Device is blocked state because of a swap.	
25	DeviceTempBlockedException	9	23	Device is blocked state temporarily.	*	No	The Device is at risk, the terminal is blocked for 10	

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
							minutes! Please try again.	
26	EncryptionKeyIsNullException	11	4	Definition error. Encryption Keys are not defined in system.	*	No	No action is needed, please contact with sys admin.	
27	HsmConnectionException	6	14	Hsm connection attemp is failed		No	No action is needed, please contact with sys admin. HSM connection permissions should be checked.	
28	GeoDistanceException	12	9	Device location is not within the limits.	*	Yes	User will redirect to Risk Management →Geo-Location Limits → Device working territory limits should be checked.	https://softpostest.paycore.com/softpos/#/limits/geol
29	HttpRequestException	2	1	Http request error.		No		
30	InvalidBinException	1	3	Bin is not permitted for this request.		No	Check the BIN definitions.	
31	InvalidSessionException	1	3	Session is not valid in system.	*	No		
32	KeyNotFoundException	11	2	Device keys are not found in ssytem.	*	No	Sys Admin should check logs.	
33	LocationRequiredException	12	12	Location information is required.		No	On the device settings, location permissions should be defined for the app.	
34	MerchantNotFoundException	3	43	Merchant information is not found in system.	*	Yes	User will redirect to Operation → Merchant Management → Click add new merchant button and new merchant will be defined.	https://softpostest.paycore.com/softpos/#/spsmerch
35	MissingRequiredTagException	4	8	Configuration error. EMV tags are not found in message.	*	Yes	User will redirect to Operation → Acq Flow Config → InOut Rule Config definitions should be checked.	https://softpostest.paycore.com/softpos/#/spsacqflo

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
36	ModelNotFoundException	9	20	Device model is not found in whitelist.		Yes	1- User will redirect to Risk Management → Awaiting Permissions → Add new device to the White List Devices. 2- User will redirect to Risk Management → White List → Add new White Listed Devices.	1- https://softpostest.paycore.com/softpos/#/awaitingP 2- https://softpostest.paycore.com/softpos/#/whitelis
37	ModelNotSupportedException	9	5	Device model is not supported from system.	*	Yes	1- User will redirect to Risk Management → Awaiting Permissions → Add new device to the White List Devices. 2- User will redirect to Risk Management → White List → Add new White Listed Devices.	1- https://softpostest.paycore.com/softpos/#/awaitingP 2- https://softpostest.paycore.com/softpos/#/whitelis

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
38	OperatingSystemNotFoundException	13	20	Operating system is not found in whitelist.			1- User will redirect to Risk Management → Awaiting Permissions → Add new OS to the White List OS. 2- User will redirect to Risk Management → White List → Add new White Listed OS.	1- https://softpostest.paycore.com/softpos/#/awaitingP 2- https://softpostest.paycore.com/softpos/#/whitelis
39	OperatingSystemNotSupportedException	13	5	Device's operating system is not supported from system.		Yes	1- User will redirect to Risk Management → Awaiting Permissions → Add new OS to the White List OS.	1- https://softpostest.paycore.com/softpos/#/awaitingP 2- https://softpostest.paycore.com/softpos/#/whitelis

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
							2- User will redirect to Risk Management → White List → Add new White Listed OS.	
40	PinDeviceKeysExpiredException	5	7	Device keys are expired.		No		
41	PublicKeyNotFoundException	11	2	Configuration error. System's public key is not found in system.	*	No		
42	RequiredParameterException	4	12	Parameter is required.		Yes	User will redirect to Operation → Acquirer Parameter → parameters will be checked or a new parameter will be added.	https://softpostest.paycore.com/softpos/#/spsacqpar
43	SafetyNetException	10	24	SafetyNet check is failed.		Yes	If the terminal gets a safety net exception, there will be 3 actions: 1- First terminal gets a safety error, therefore it will be at risk! 2- If the safety net API call limit exceeds, close the safety net control, or try again. 3- If a specific white list device gets that safety net exception, Risk Management → White List → SafetyNet Support status will be false.	https://softpostest.paycore.com/softpos/#/whitelist
44	SessionMissingException	1	1	Session is null or empty.	*	No	No action is needed.	
45	SessionNotFoundException	1	2	Session is not found in system.	*	No	Please contact with sys admin!	
46	~~SettlementNullResponseException~~	2	1	Settlement response is null.		No	No action is needed.	

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
47	~~TerminalKeyEmptyException~~	11	8	At least one of acquirer shared terminal key is empty.	*	No	No action is needed.	
48	~~TerminalKeyIsNullException~~	11	4	At least one of acquirer shared terminal key is null.	*	No	No action is needed.	
49	~~TerminalKeysetParameterConfigException~~	11	8	Configuration error. Acquirer terminal keyset parameter is not found in system.	*	No	No action is needed.	
50	TerminalNotFoundException	3	48	Terminal is not found in system.		Yes	Operation → Terminal Management → Terminal data will be checked or a new terminal will be added.	https://softpostest.paycore.com/softpos/#/spstermin
51	TerminalParametersChangedException	9	6	Acquirer terminal parameters has been changed.	"Business parameter update required."		Operation→ Terminal Profile Management → Business Parameter will be updated.	https://softpostest.paycore.com/softpos/#/terminalpr
52	TransactionNotFoundException	2	2	Emv transaction is not found in system.	*	No	Sys Admin should check the Logs and Database.	
53	TransactionTimeoutException	2	10	Emv transaction is expired for system.	*	No	Sys Admin should check the Logs and Database.	
54	VoidTransactionTimeException	4	4	Void transaction time period(1 day) is over.	`Can not send void transaction if it is not performed on same day with original transaction!`	No	No action is needed	
55	DeviceFirmwareUpdatedException	9	13	Device firmware is updated and build number is changed, user needs to re-register.	"Device fingerprint is not matched with device information."	No	The terminal owner should be re-register	
56	VoidWithWrongCardException	4	27	Void transaction is not performed with same card on original transaction.		No	Error description should be considered first, otherwise, Sys Admin should check the Logs and Database.	
57	AlreadyVoidedException	4	4	Void transaction request has been sent more than once for same transaction.		No	Error description should be considered first, otherwise, Sys Admin should check the Logs and Database.	
58	BdkKeyNotFoundException	11	28			No	Sys Admin should check the Logs and Database.	
59	SSLPeerUnverifiedException	15	15	Certificate pinning failure		No	No action is needed.	

#	Exception Name	Error Code	Error SubCode	Description	Example (Description)	Actionable	Actions' Description	Action URL
60	TerminalDisabledException	3	47	Terminal is not allowed to perform transaction		Yes	Operation → Terminal Management → IsEnableTerminal column should be checked.	https://softpostest.paycore.com/softpos/#/spstermin
61	MerchantDisabledException	3	46	Merchant is not allowed to perform transaction			Operation → Merchant Management → IsEnableMerchant column should be checked.	https://softpostest.paycore.com/softpos/#/spsmerch

#	Error Code	Error Name	Http Status Code
1	1	Security	401
2	2	Runtime	409
3	3	Configuration	424
4	4	Unexpected	500
5	5	Pin	409
6	6	Hsm	409
7	7	AcquirerHost	409
8	8	Database	409
9	9	Device	409
10	10	Attestation	409
11	11	Key	409
12	12	Location	409
13	13	DeviceOS	409

#	Error SubCode	Error Name
1	1	Unexpected
2	2	NotFound
3	3	Invalid
4	4	InvalidData
5	5	NotSupported
6	6	Changed
7	7	Expired
8	8	Missing
9	9	SecurityRisk
10	10	Timeout
11	11	Blocked
12	12	Required
13	13	FirmwareUpdate
14	14	NotConnected
15	15	PaymentLimit
16	16	CountLimit
17	17	TimeLimit
18	18	Failed
19	19	Format
20	20	Whitelist
21	21	Version
22	22	Swap
23	23	TemporaryBlocked
24	24	SafetyNet

25	25	PackageName
26	26	Permission
27	27	WrongCard
28	28	Unregister
29	29	Transaction Time Limit
30	30	Package Name
31	31	App Version
32	32	Permission
33	33	Device Model
34	34	Memory Percentage
35	35	Remaining Battery Percentage
36	36	Available External Storage Percentage
37	37	Available Internal Storage Percentage
38	38	OS Name And OS Version
39	39	Device Model And Manufacturer
40	40	CertificateDigestSha256
41	41	Device IMEI
42	42	Geo Attestation Check
43	43	Merchant Not Found
44	44	Batch Upload Needed Config Not Found
45	45	Blacklist
46	46	Merchant Disabled
47	47	Terminal Disabled
48	48	Terminal Not Found
49	49	Invalid Activation Code
50	50	Already Voided
51	51	Phone Number Not Found
52	52	`Otp Not Send`
53	53	Activation Code Save Error

16.MF Qr Transaction Flow Implementation

This method is used to send qr transaction requests to SoftPos backend on secure channel.

Qr Generation

```
public <E, T> void sendCustomRequest(final String path, E request, final Class responseClassType, String mti, final
```

No	Value	MTI Reference
1	0100	Sale
2	0200	Void
3	0300	Refund
4	0900	Receipt
5	0980	CustomData
6	0405	Reversal
7	0600	PreAuth
8	0905	Qr Generation
9	0915	Qr Inquiry
10	0925	Qr Sale
11	0935	Qr Refund / Partial
12	0945	Qr Void
13	0955	Qr Reversal
14	0965	FastSale
15	0975	FastRefund

No	Id	Cvm Type
1	33	No cvm performed
2	32	Signature
3	31	PIN verified

16.1 QR Generation

Request Parameters

```
Map<String, String> message = new HashMap<>();
message.put("qrAmount","000000001000");
message.put("qrTxnType","00");
message.put("qrInstCount","");
message.put("qrFastTxnMsgNumber","");
message.put("qrLengthType","1");
message.put("originalTransactionId","");
```

Parameter Name	Parameter Type	Desciption	
`qrAmount`	Mandatory	Transaction amount value	000000001000 for 10,00

<table>
<tr><th>Parameter Name</th><th>Parameter Type</th><th>Desciption</th><th></th></tr>
<tr><td>qrTxnType</td><td>Mandatory</td><td><table><tr><th>Value</th><th>Type</th></tr><tr><td>00</td><td>None</td></tr><tr><td>01</td><td>Sale</td></tr><tr><td>02</td><td>Installment Sale</td></tr><tr><td>03</td><td>Cancel</td></tr><tr><td>04</td><td>Refund</td></tr><tr><td>05</td><td>Cash Advance</td></tr><tr><td>06</td><td>Point Sale</td></tr><tr><td>07</td><td>Fast Refund</td></tr></table></td><td></td></tr>
<tr><td>qrInstCount</td><td>Optional</td><td></td><td></td></tr>
<tr><td>qrFastTxnMsgNumber</td><td>Optional</td><td></td><td></td></tr>
<tr><td>qrLengthType</td><td>Mandatory</td><td><table><tr><th>Value</th><th>Type</th></tr><tr><td>1</td><td>Short QR</td></tr><tr><td>2</td><td>Long QR</td></tr></table></td><td></td></tr>
<tr><td>retrievalTransactionId</td><td>Optional</td><td>Transaction Id</td><td></td></tr>
</table>

Response Parameters

```
Map<String, String> response = new HashMap<>();
response.get("qrData");
response.get("qrRefNo");
response.get("f38");
response.get("f39");
response.get("qrTransactionId")
```

QR Data Example


```
303030323031303130323132323638303030313054522E434F4F4D2E424B4D303630313130383332393634353 73
939353044413939423433313843354136333037463142304441413039313054445 64D41554A30303031303031
4E3131303230303330393230303136545 22E474F562E54434D422E46415354303132365452393330303231303
030303030303039353030323030303031303230323031323033323936343537393935304441393942343331 38
433541363330374631423044414134393130303038393034313033363531383230303032313030320343032 3
130303331323939393939393939383435333034303230323035313030303030313932303937303631323232 30
333138313634393433303731323232303331383138343934333532303430303030353330333934393534313 23
030303030303030313030303538303254523539323248495A49524FC49E4C5520C4B04EC59E41415420544141
482E36303038C4B05354414E42554C36313035333430303036323330303231313430303030303030303030303
630353935303032303830323037363433373030303254523031313557534 44C2044454E454D45205550443032
3038C4B05354414E42554C3633303430453139
```


Call Method

Sample Implementation Code

```
public void QrGeneration(Map<String, String> message,  MtiType  mtiType , CustomRequestListener<Map<String, String>
    this.softPosService.sendCustomRequest("qrTransactionAsync", mtiType.getValue() ,message, Hashmap.class, listene
}
```

16.2 Qr Inquiry

Inquiry Request

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId","100004892005");
message.put("qrFlowType",0);
```

Inquiry Result

```
Map<String, String> response = new HashMap<>();
response.get("qrData");
response.get("qrRefNo");
response.get("f38");
response.get("f39");
```

QR Data Example


```
303030323031303130323132323638303030313054522E434F4D2E424B4D303630313130383332393634353 73
939353044413939423433313843354136333037463142304441413039313054444564D41554A30303031303031
4E313130323030333039323030313654522E474F562E54434D422E46415354303132365452393330303231303
030303030303039353030323030303031303230323031323033323936343537393935304441393942343331 38
433541363330374631423044441413439313030303839303431303336353138323030303231303032303430323
130303331323939393939393939383435333034303230323035313030303030313932303937303631323232 30
333138313634393433303731323232303331383138343934333532303430303030353330333934393534313 23
030303030303030313030303538303254523539323248495A49524FC49E4C5520C4B04EC59E41415420544141
482E36303038C4B05354414E42554C36313035333430303036323330303231313430303030303030303030303
630353935303032303830323037363433373030303254523031313557534 44C2044454E454D45205550443032
3038C4B05354414E42554C3633303430453139
```


Call Method

Sample Implementation Code

```
public void QrCustomRequest(Map<String, String> message, MtiType  mtiType, CustomRequestListener<Map<String, String
    this.softPosService.sendCustomRequest("qrTransactionAsync", mtiType.getValue() ,message, Hashmap.class, listene
}
```

Custom Data

Custom message field in transaction request. In inquiry ,sale ,void and refund,reversal transaction please send transactionId field in customData parameter.

Custom Data Parameters		
Parameter Name	**Parameter Type**	**Desciption**
`mti`	Mandatory	Mti Value
`qrPinData`	Optional	Pin Data
`qrAmount`	Optional	Transaction amount value
`qrFlowType`	Mandatory	Qr Type

Result Methods

When transaction is completed, onCompleted method will be triggered. Result of the transaction can be received by using TransactionResult methods.

Inquiry Response

```
public class TransactionResult {


private boolean emvAccepted;

private KernelResultType kernelResult;

private boolean withPin;
```

```
private String refusalCode;

private String maskedPan;

private CardType cardType;

private String issuerIdentificationNumber;

private Map<String,String> customMessage;

private String outcomeMsgId;

private String l2ErrorIndication;

private String transactionId;

private String applicationPreferredName;

private String applicationLabel;

private PinResult pinResult;

private SoftPosError transactionError;

private boolean isSCANeeded;

private boolean isQRTransaction;

}
```

Parameter Name	Description	Sample Value
isEmvAccepted	L2 Kernel has completed the transaction successfully.	true or false
getRefusalCode	The authorization result of the transaction	51 - (insufficient fund)
getMaskedPan	Last 4 digit of the card	7204
getCardType	Kernel Id value of the transaction	CardType.MasterCard
getIssuerIdentificationNumber	Issuer Identification Number of card	534567
getCustomMessage	Response message of your custom message in transaction request	
getL2ErrorIndication	Kernel error indication	00
getTransactionId	The unique identifier of the transaction.	929616000304
getApplicationLabel	The application label field of the card. It can be empty depending on card.	Debit MasterCard
getApplicationPreferredName	The application preferred name of the card. It can be empty depending on card.	NEO
isSCANeeded	Start new transaction for SCA re-tap workflow. Please check section 5.9 Strong Customer Authentication	true or false
getTransactionError	If you get a system error which refusal code is 96, you can get the error details with using this method. Please check 8. SoftPos Error section for the details. It will be null if the authorization host return 96 value.	

Transaction Listener Methods

Method Name	Description	Action	Sample POS Led Design
onCompleted	It is triggered when SDK has finished transaction.	Check TransactionResult and inform user that transaction has been accepted or failed.	
onTimeout	It is triggered when SDK could not detect any cards in a given timeout value	Inform user that mobile app could not detect any cards. Transaction is cancelled.	
onStartTransactionError	It is triggered when SDK gets an error during starting transaction	Check the SoftPosError object to understand the error.	

16.3 Qr Sale

Sale Request

```
String qrTransactionId = "100004892005"

Map<String, String> customData = new HashMap<>();
customData.put("qrPinData","");
customData.put("mti", mtiType.QrSale.getValue());
customData.put("qrFlowType", 0);
```

Sale Response

```
public class TransactionResult
```

Call Method

Sample Implementation Code

```
public void startQRTransaction(final String qrTransactionId, final Map<String, String> customData, final Transactio

    this.softPosService.startQRTransaction(qrTransactionId, customData, transactionListener);
}
```

16.4 Qr Refund / Partial Refund

Refund Request

```
String qrTransactionId = "100004892005"


Map<String, String> customData = new HashMap<>();
customData.put("mti",  mtiType.QrRefund.getValue());
customData.put("qrAmount","000000001000");
customData.put("qrFlowType", 0);
```

Refund Response

```
public class TransactionResult
```

Call Method

Sample Implementation Code

```
public void startQRTransaction(final String qrTransactionId, final Map<String, String> customData, final Transactio

    this.softPosService.startQRTransaction(qrTransactionId, customData, transactionListener);
}
```

16.5 Qr Void

Void Request

```
String qrTransactionId = "100004892005"

Map<String, String> customData = new HashMap<>();
customData.put("mti", mtiType.QrVoid.getValue());
customData.put("qrFlowType", 0);
```

Void Response

```
public class TransactionResult
```

Call Method

Sample Implementation Code

```
public void startQRTransaction(final String qrTransactionId, final Map<String, String> customData, final Transactio

    this.softPosService.startQRTransaction(qrTransactionId, customData, transactionListener);
}
```

16.6 Qr Reversal

Reversal Request

```
String qrTransactionId = "100004892005"

Map<String, String> customData = new HashMap<>();
customData.put("mti", mtiType.QrReversal.getValue());
customData.put("qrFlowType", 0);
```

Reversal Response

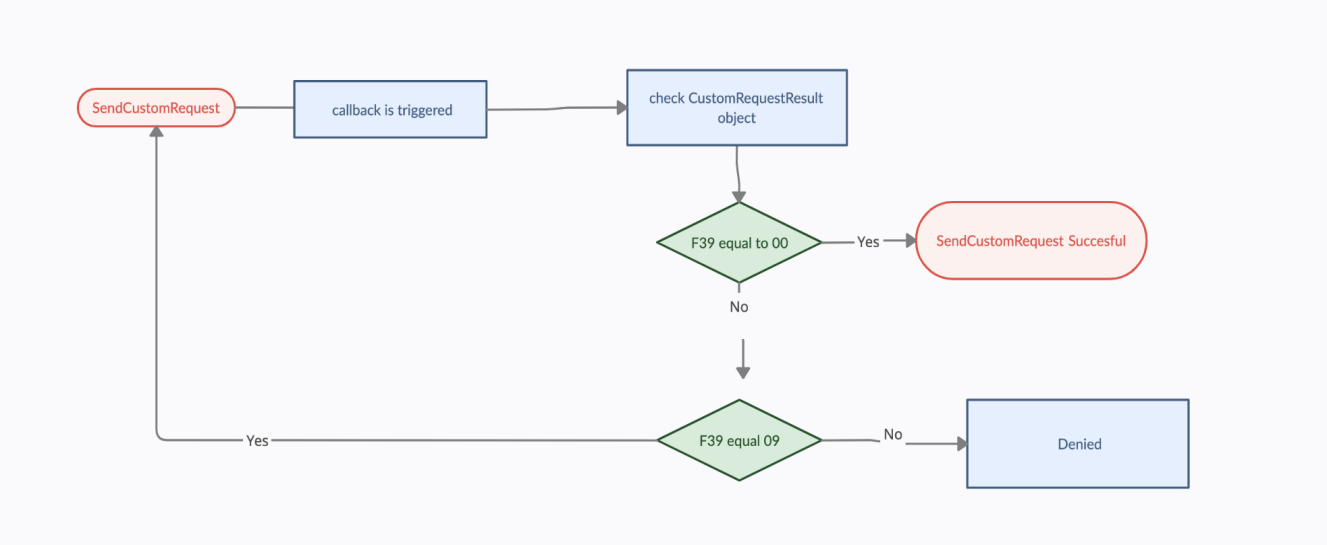
```
public class TransactionResult
```

Call Method

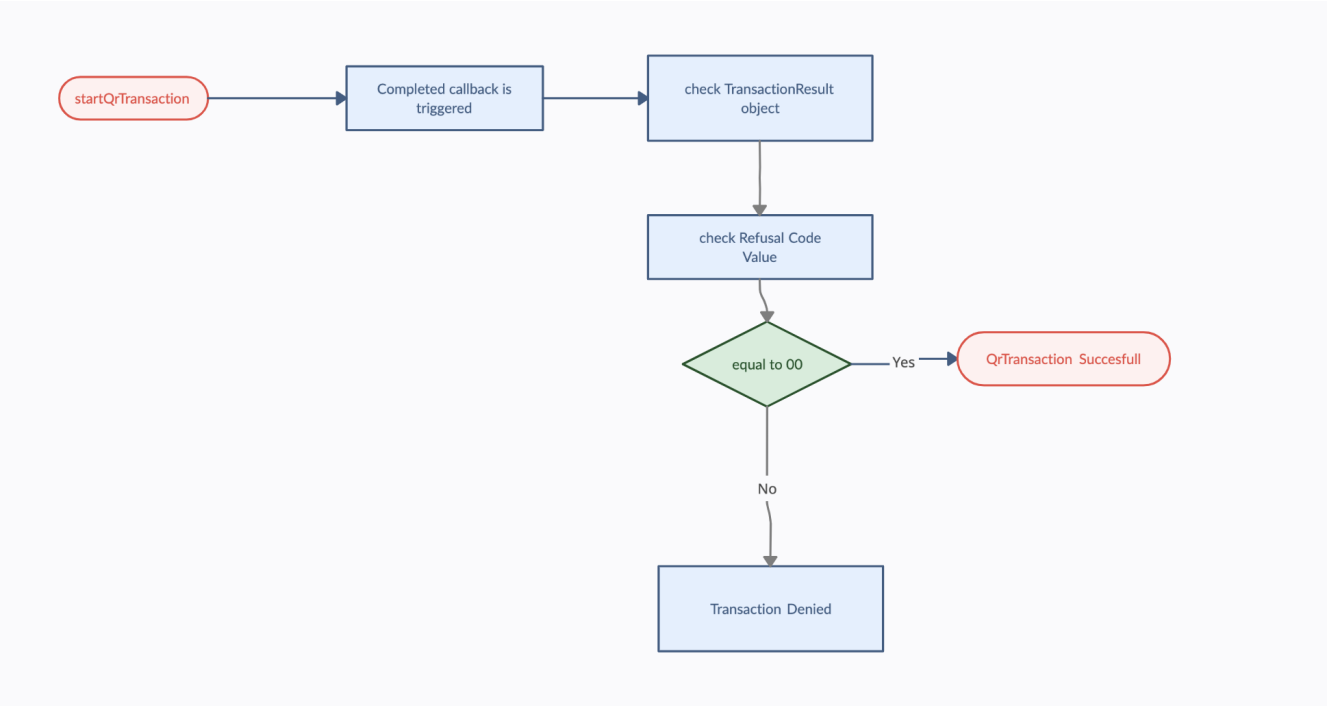
Sample Implementation Code

```
public void startQRTransaction(final String qrTransactionId, final Map<String, String> customData, final Transactio

    this.softPosService.startQRTransaction(qrTransactionId, customData, transactionListener);
}
```

Custom Request Diagram



QR Transaction Diagram



17. QR Flow

This method is used to send qr transaction requests to SoftPos backend on secure channel.

Qr Generation

No	Value	MTI Reference
1	0100	Sale
2	0200	Void
3	0300	Refund
4	0900	Receipt
5	0980	CustomData
6	0405	Reversal
7	0600	PreAuth
8	0905	Qr Generation
9	0915	Qr Inquiry
10	0925	Qr Sale
11	0935	Qr Refund / Partial
12	0945	Qr Void
13	0955	Qr Reversal
14	0965	FastSale
15	0975	FastRefund

QRCustomResponse: "f133" field in the response map

Tag	Type	
qrData	String	QRGenerationRequest
qrRefNo	String	QRGenerationRequest
f38	String	Any request
f39	String	Any request
qrTransactionId	String	All custom request except QRGenerationRequest
cvmType	String	QRInquiryRequest
fastTransaction	String	QRInquiryRequest, Decide fast transaction or normal transaction according to "true" and "false" values
isoStan	String	Any request

No	Id	Cvm Type
1	33	No cvm performed
2	32	Signature
3	31	PIN verified

QR Custom Request

<table>
<tr><th>Parameter Name</th><th>Parameter Type</th><th>Desciption</th><th></th></tr>
<tr><td>qrAmount</td><td>Mandatory</td><td>Transaction amount value</td><td>000000001000 for 10,00</td></tr>
<tr><td>qrTxnType</td><td>Mandatory</td><td><table><tr><th>Value</th><th>Type</th></tr><tr><td>00</td><td>None</td></tr><tr><td>01</td><td>Sale</td></tr><tr><td>02</td><td>Installment Sale</td></tr><tr><td>03</td><td>Cancel</td></tr><tr><td>04</td><td>Refund</td></tr><tr><td>05</td><td>Cash Advance</td></tr><tr><td>06</td><td>Point Sale</td></tr><tr><td>07</td><td>Fast Refund</td></tr></table></td><td></td></tr>
<tr><td>qrInstCount</td><td>Optional</td><td></td><td></td></tr>
<tr><td>qrFastTxnMsgNumber</td><td>Optional</td><td></td><td></td></tr>
<tr><td>qrLengthType</td><td>Mandatory</td><td><table><tr><th>Value</th><th>Type</th></tr><tr><td>1</td><td>Short QR</td></tr><tr><td>2</td><td>Long QR</td></tr></table></td><td></td></tr>
<tr><td>originalTransactionId</td><td>Optional</td><td>Orginal Transaction Id</td><td></td></tr>
</table>

Request Type:

Custom Request

Request:

message: Map<String, String>
mti: string
customRequestListener: CustomRequestListener<Map<String, String>>

Response:

response: Map<String, String>
softPosService.sendCustomRequest("qrTransactionAsync", message, mti, new CustomRequestListener<Map<String, String>>(){
onGetCustomResponse
onCustomRequestError
});

StartQRTransaction

Request:

requestData: Map<String, String>
qrTransactionId: String
transactionListener: TransactionListener

SoftPosApplication.getSoftPosUtil().getSoftPosService().startQRTransaction(qrTransactionId, requestData, new TransactionListener(){
onCompleted

..someCallbackFunctions

})

Response:

transactionResult: (onCompleted)
CheckPosService

Request:

```
SoftPosApplication.getSoftPosUtil().checkPOSService(new CheckPOSServiceListener(){
onCheckPOSSuccess
onCheckPOSError
});
```

Response:

void

Transaction Flow

Note: All requests must be waiting for a response from the predecessor request before being sent

QR Sale Request:

PreRequisete:

After the inquiry request, if fastTransaction is "false" the response then call QR Sale request

Flow (All requests must be waiting response for predecessor request)

QR Generation Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
String amount = "000000000550";
message.put("qrTxnType", "01");
message.put("qrAmount", amount);
message.put("qrInstCount", "00");
message.put("qrFastTxnMsgNumber", "");
message.put("qrLengthType", "1");
mti: MtiType.QrGeneration
```

Response Parameters:

response: Map<String, String>
response.get("f133"): QRCustomResponse

QR Inquiry Request

Function: sendCustomRequest

Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
```

Response Parameters:

response: Map<String, String>
response.get("f133"): QRCustomResponse

Logic:

if f39 is an error state then call the inquiry request until getting success response.

QR Sale Request

Function: checkPOSService
Request Parameters:

Respose Parameters: wait to success response

Function: startQRTransaction

Request Parameters:

```
Map<String, String> customData = new HashMap<>();
customData.put("cvmType", cvmType);
customData.put("transactionAmount", amount);
customData.put("mti", MtiType.QrSale.getValue());
```

Response Parameters:

transactionResult: TransactionResult
Note: cvmType is 31 that given by inquiry, startQRTransaction service will be starting pin application for pin flow

QR Void Flow:

PreRequisete:
For void list menu, if selected item for "mti" field is 0925, call this service

QR Generation Request

Function: sendCustomRequest

Request Parameters:

```
Map<String, String> message = new HashMap<>();
String amount = "000000000550";
message.put("qrTxnType", "03");
message.put("qrAmount", amount);
message.put("qrInstCount", "00");
message.put("qrFastTxnMsgNumber", "");
message.put("originalTransactionId", originalTransactionId);
message.put("qrLengthType", "1");
mti: MtiType.QrGeneration
```

Response Parameters:

```
response: Map<String, String>
response.get("f133"): QRCustomResponse
```

QR Inquiry Request

Function: sendCustomRequest

Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
Response Parameters:
response: Map<String, String>
response.get("f133"): QRCustomResponse
```

Logic:

if f39 is an error state then call the inquiry request until getting success response.

QR Void Request

function: sendCustomRequest

Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
```

Response Parameters:

```
response: Map<String, String>
response.get("f133"): QRCustomResponse
```

QR Refund Flow

PreRequisete:
For refund list menu, if selected item for "mti" field is 0925, call this service

QR Generation Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
String amount = "000000000550";
message.put("qrTxnType", "05");
message.put("qrAmount", amount);
message.put("qrInstCount", "00");
message.put("qrFastTxnMsgNumber", "");
message.put("originalTransactionId", originalTransactionId);
message.put("qrLengthType", "1");
mti: MtiType.QrGeneration
```

Response Parameters:

response: Map<String, String>
response.get("f133"): QRCustomResponse

QR Inquiry Request

function: sendCustomRequest

Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");

Response Parameters:
response: Map<String, String>
response.get("f133"): QRCustomResponse
```

Logic:

if f39 is an error state then call the inquiry request until getting success response.

QR Refund Request

function: sendCustomRequest

Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
```

Response Parameters:

```
response: Map<String, String>
response.get("f133"): QRCustomResponse
```

QR Fast Sale Flow

PreRequisete:

After the inquiry request, if fastTransaction is "true" the response then call QR Fast Sale request

QR Generation Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
String amount = "000000000550";
message.put("qrTxnType", "01");
message.put("qrAmount", amount);
message.put("qrInstCount", "00");
message.put("qrFastTxnMsgNumber", "");
message.put("qrLengthType", "1");
mti: MtiType.QrGeneration
```

Response Parameters:

```
response: Map<String, String>
response.get("f133"): QRCustomResponse
```

QR Inquiry Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
```

Response Parameters:

response: Map<String, String>
response.get("f133"): QRCustomResponse

Logic:

if f39 is an error state then call the inquiry request until getting success response.

QR Fast Sale Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
```

Response Parameters:

response: Map<String, String>
response.get("f133"): QRCustomResponse

QR Fast Refund Flow

PreRequisete

For refund list menu, if selected item for "mti" field is 0965, call this service
For void list menu, if selected item for "mti" field is 0965 , call this service

QR Generation Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
String amount = "000000000550";
message.put("qrTxnType", "05");
message.put("qrAmount", amount);
message.put("qrInstCount", "00");
message.put("qrFastTxnMsgNumber", "");
message.put("originalTransactionId", originalTransactionId);
message.put("qrLengthType", "1");
mti: MtiType.QrGeneration
```

Response Parameters:

```
Map<String, String>
response.get("f133"): QRCustomResponse
```

QR Inquiry Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
mti: MtiType.QrGeneration
Response Parameters:
response: Map<String, String>
response.get("f133"): QRCustomResponse
```

Logic:
if f39 is an error state then call the inquiry request until getting success response.

QR Fast Refund Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
mti: MtiType.QrGeneration
```

Response Parameters:

response: Map<String, String>
response.get("f133"): QRCustomResponse

QR Reversal Flow

PreRequisete

if there is any interruption after just the QR generation request then call the reversal request

QR Generation Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
String amount = "000000000550";
message.put("qrTxnType", "05");
message.put("qrAmount", amount);
message.put("qrInstCount", "00");
message.put("qrFastTxnMsgNumber", "");
message.put("qrLengthType", "1");
mti: MtiType.QrGeneration
```

Response Parameters:

```
Map<String, String>
response.get("f133"): QRCustomResponse
```

QR Inquiry Request

Function: sendCustomRequest
Request Parameters:

```
Map<String, String> message = new HashMap<>();
message.put("qrTransactionId", qrTransactionId);
message.put("qrFlowType", "0");
```

Response Parameters:

```
response: Map<String, String>
response.get("f133"): QRCustomResponse
```

18. MPoC Only API's

This section is about defining API's which are only available in integration with MPoC SDK.

1. StartManualEntryTransaction API

This API is used for starting manual entry transaction. An input screen is going to be opened to be able to get PAN, Expire Date and CVV instead of NFC card read screen.

Manual Pan screen will run on different process.

Explanation of API parameters are like in the table below;

startManualEntryTransaction

```
public void startManualEntryTransaction(final long amount, final TransactionType transactionType, final int timeout
```

<table>
<tr><th>Parameter</th><th>Description</th><th>Sample Value</th></tr>
<tr><td>amount</td><td>Amount of transaction</td><td>Set 1300 for 13 €</td></tr>
<tr><td>transactionType</td><td>Emv parameter of the transaction</td><td>• SALE
• REFUND
• VOID</td></tr>
<tr><td>timeoutMs</td><td>Timeout value to tap the card to the mobile device.</td><td>10000 ms</td></tr>
<tr><td>mti</td><td>Type of Transaction (Sale, Void vs)</td><td><table><tr><th>Type</th><th>Value</th></tr><tr><td>Sale</td><td>0100</td></tr><tr><td>Void</td><td>0200</td></tr><tr><td>Refund</td><td>0300</td></tr><tr><td>Void Cancellation</td><td>0400</td></tr><tr><td>Pre Authorization</td><td>0500</td></tr></table></td></tr>
<tr><td>customMessage</td><td>Custom message field in transaction request</td><td>In void and refund transaction please send transactionId field in customMessage parameter.
TransactionID
<pre>Map<String, String> customMessage = new HashMap<>();

message.put("transactionId", "17823678");</pre></td></tr>
<tr><td>transactionListener</td><td>Asynchronous caller function.</td><td></td></tr>
</table>

19. Device Custom Parameter (DCP)

DCP is a method to proceed with custom transactions next to main transactions like sales. DCP is now enabled to use for Installment, Point Sales, and Discounts on sale transactions any of them can be used under DCP.

sendCustomRequest

```
public <E> void sendCustomRequest
(final E request, final String base, final String path, final String authHeader, final OperationListener calll
```

Table Custom Request Parameters

Parameter Name	Description	Sample Value
request	Type of request	`MethodIDType.SEND_CUSTOM_REQUEST`
path	In order to make request to custom sub domains, set path value. It is added to the main url which is set via SoftPosInfo.setUrl method.	“MethodIDType.SEND_CUSTOM_REQUEST“
request	Instance of request class. The class should have @Json annotation for related field. Note: To send raw json, set json string.	Check Example 1 Request Class
responseClassType	Class type of response object. It is required for deserialize response to given class type. Note: To get response as a json string. Set String.Class	ListResponse[].class
customRequestListener<T>	Asynchronous caller function. Note: To get response as a json string. Set CustomRequestListener<String>	

Example 1 Request Class

The request class is the same as a Start Transaction method. 5. Start Transaction

When the transaction started, according to core configuration DCP response will return to SDK after that parsing FinancialTxnResponse from Transaction Result Object.

Example 2 Response Class

FinancialTxnResponse

```
public class FinancialTxnResponse {

    @Json(name = "emvElements")
    private List<InOutElement> emvElements;

    public FinancialTxnResponse(List<InOutElement> emvElements) {
        this.emvElements = emvElements;
    }

    public List<InOutElement> getEmvElements() {
        return emvElements;
    }

    public void setEmvElements(List<InOutElement> emvElements) {
        this.emvElements = emvElements;
    }

    public InOutElement findEmvElementBy(String tag){
        for (InOutElement element:emvElements) {
            if(element.getTag().equals(tag)){
```

```
                return element;
            }
        }
        return null;
    }
}
```

Device Custom Parameter Object

```
public class DeviceCustomParameter {

    @Json(name = "discount")
    private String discount;

    @Json(name = "installment")
    private String installment;

    @Json(name = "point")
    private String point;

    @Json(name = "maxInstallmentCount")
    private String maxInstallmentCount;

    @Json(name = "txnId")
    private String txnId;
}
```

DCP Output Element

```
output elements:
[
{"tag":"f133",
"value":"{"DCP":"{"installment":"3,","maxInstallmentCount":"120","point":"true","txnId":"319344622021"}",
"isoStan":"001913","f39":"DCP"}",
"fieldFormat":null}
]
```

Custom Request Listener

The response of the SEND_CUSTOM_REQUEST method can be reached using sendDcp methods. An exemplary usage is below:

CustomRequestListener

```
public interface CustomRequestListener<T> {

    void onGetCustomResponse(T response);

    void onCustomRequestError(SoftPosError error);
}
```

Table Custom Request Listener Methods

Method Name	Description	Action
onGetCustomResponse	Response is successfully gotten.	
onCustomRequestError	When SDK encounters unhandled error during sendCustomRequest call, this method will be triggered with the error message.	Get error detail using SoftPosError class. Please check 8. SoftPos Error section.

sendDcp method is a caller method for sendCustomRequest method

```
public void sendDcp(Map<String, String> dcp, CustomRequestListener<TransactionResult> listener) {
    this.softPosService.sendCustomRequest("transactionAsync", dcp, Mti.DCP.getValue(), listener);
}
```

Installment send custom message example

```
{"DCP":"{"installment":"3,","maxInstallmentCount":"","point":"","txnId":"319344622021"}}
```

sendDcp method usage example

```
SoftPosApplication.getSoftPosUtil().sendDcp(customMessage, new CustomRequestListener<TransactionResult>() {
 @Override
 public void onGetCustomResponse(TransactionResult response) {

 }

 @Override
 public void onCustomRequestError(SoftPosError softPosError) {

 }
});
```